ZERO TO HERO / PREMIUM HANDWRITTEN NOTES

Microservices

Java + Spring Boot

From your first REST API to resilient, interview-ready distributed architectures.

120 ORIGINAL QUESTIONS

120 FOLLOW-UP ANSWERS

14 ARCHITECTURE SKETCHES

- ✦ 24 CHAPTERS / FUNDAMENTALS TO ARCHITECT
- ✦ 10 JAVA LABS + 2 PROJECT RECIPES
- ✦ REAL FAILURE PATHS / 30-DAY PRACTICE

THE NOTEBOOK

How to get interview-ready

Read the question. Answer it out loud in 30–60 seconds before opening the explanation. Follow the arrows, study the example, and answer the follow-up without guessing the failure behavior.

120	120	14
Unchanged questions	New follow-up answers	Architecture sketches

Use one interview answer pattern

Requirement → owner/invariant → design → crash or timeout → recovery → evidence.

Practice before memorizing

The first 10 labs are Java 17 local models. The last two labs are PostgreSQL and Spring integration recipes. Local tests do not prove the live distributed guarantees.

Java & framework boundaries

Runnable Java models target Java 17. Spring client/security examples depend on compatible Spring versions and external infrastructure. Example tokens, addresses and identifiers are placeholders.

FIND YOUR CHAPTER

24 topics / 120 questions

01. Microservices foundations Q001–Q005

02. Domain boundaries and ownership Q006–Q010

03. REST contracts and API evolution Q011–Q015

04. Communication and Java clients Q016–Q020

05. Discovery, gateways and service mesh Q021–Q025

06. Spring Boot implementation and configuration Q026–Q030

07. Timeouts, deadlines and retries Q031–Q035

08. Resilience and admission control Q036–Q040

09. Data ownership and consistency Q041–Q045

10. Concurrent writes and data invariants Q046–Q050

11. Sagas and workflow recovery Q051–Q055

12. Transactional outbox and idempotent consumers Q056–Q060

13. Events, schemas and Kafka topics Q061–Q065

14. Delivery semantics, consumer groups and replay Q066–Q070

15. CQRS, projections and caching Q071–Q075

16. Security across service boundaries Q076–Q080

17. Observability and reliability objectives Q081–Q085

18. Testing distributed contracts and failures Q086–Q090

19. Deployment, Kubernetes and schema changes Q091–Q095

20. Capacity, performance and regional design Q096–Q100

21. Migration and architecture evolution Q101–Q105

22. Business architecture case studies Q106–Q110

23. Production diagnosis and recovery Q111–Q115

24. Senior and architect interview judgment Q116–Q120

Extra: 14 architecture diagrams · 10 compiled local Java models · 2 real-boundary recipes · 30-day practice plan · quick revision.

THE ARCHITECT INTERVIEW NOTEBOOK

PART I

FOUNDATIONS & API CONTRACTS

Agree who owns the work, the data and the HTTP contract.

Question 001-030

CHAPTER 01/24

Microservices foundations

Own one business capability rather than a collection of URLs.

Five interview questions

Q001. What is a microservice?

Q002. How do a monolith, modular monolith and microservices differ?

Q003. What is a distributed monolith?

Q004. What are the benefits and costs of microservices?

Q005. When should you avoid introducing microservices?

Your challenge

Explain one correct operation, one failure path and the evidence that would tell you the design is working.

Microservices foundations

Q001. What is a microservice?

THE 30-SECOND ANSWER

A microservice handles one clear business job and can be released without forcing unrelated services to change. It owns its data and exposes a contract.

CONCEPT → REAL EXAMPLE → RISK

A microservice is an independently deployable service responsible for a... → An order service accepts orders and exposes order status → A service boundary introduces network failures and separate state

EXAMPLE FROM THE ORIGINAL GUIDE

An order service accepts orders and exposes order status. Inventory reserves stock through its own contract. Either can release a compatible implementation without forcing the other to deploy.

INTERVIEWER'S FOLLOW-UP

What evidence proves independent deployment?

Answer → Deploy Orders alone while Inventory and Payments keep running. Verify old and new API versions work together, and no shared database migration forces a simultaneous release.

COMMON INTERVIEW TRAP: A service boundary introduces network failures and separate state.

MY ONE-LINE RECAP _____

Microservices foundations

Q002. How do a monolith, modular monolith and microservices differ?

THE 30-SECOND ANSWER

A monolith ships as one application. A modular monolith has clean modules but still ships together. Microservices run and deploy separately over network contracts.

CONCEPT → REAL EXAMPLE → RISK

A monolith is deployed as one application → A loan application can begin with origination, underwriting and disbursement... → A monolith can be maintainable and horizontally scalable

EXAMPLE FROM THE ORIGINAL GUIDE

A loan application can begin with origination, underwriting and disbursement modules in one Boot application. Extract underwriting only when its release, ownership or scaling needs justify a remote boundary.

IMPLEMENTATION SKETCH *illustrative*

```
// A modular monolith can preserve domain modules:
orders.place(order);  // same deployment
// Extract only if ownership justifies remote calls.
```

INTERVIEWER'S FOLLOW-UP

Which module would you extract first, and why?

Answer → Extract a stable, high-change module such as underwriting only when ownership, release pace or scaling differs. First measure call frequency and dependency coupling.

COMMON INTERVIEW TRAP: A monolith can be maintainable and horizontally scalable.

MY ONE-LINE RECAP _____

Microservices foundations

Q003. What is a distributed monolith?

THE 30-SECOND ANSWER

A distributed monolith has many running services but still needs joint releases or shares internal tables. It gets network problems without independent change.

CONCEPT → REAL EXAMPLE → RISK

It is a collection of processes that still requires coordinated changes or... → Order, pricing and customer services all read the same database tables and must... → Counting services does not measure architectural independence

EXAMPLE FROM THE ORIGINAL GUIDE

Order, pricing and customer services all read the same database tables and must deploy together after a column rename. Separate containers have not removed their release coupling.

INTERVIEWER'S FOLLOW-UP

How would you remove one source of coupling?

Answer → Stop one direct-table dependency: expose the data through its owning service API, migrate readers, enforce separate credentials and verify independent deployment.

COMMON INTERVIEW TRAP: Counting services does not measure architectural independence.

MY ONE-LINE RECAP _____

Microservices foundations

Q004. What are the benefits and costs of microservices?

THE 30-SECOND ANSWER

Microservices can give teams ownership and allow targeted scaling, but add network errors, data coordination and monitoring work.

CONCEPT → REAL EXAMPLE → RISK

Useful benefits include focused ownership, selective scaling and... → A heavily used search service scales separately from checkout → More replicas do not repair a shared dependency bottleneck

EXAMPLE FROM THE ORIGINAL GUIDE

A heavily used search service scales separately from checkout. That benefit must pay for search-index freshness, event recovery, monitoring and a team that can operate it.

INTERVIEWER'S FOLLOW-UP

How would you measure whether the split helped?

Answer → Compare release frequency, lead time, incident rate, p95 latency, operating costs and team effort before and after the split under comparable traffic.

COMMON INTERVIEW TRAP: More replicas do not repair a shared dependency bottleneck.

MY ONE-LINE RECAP _____

Microservices foundations

Q005. When should you avoid introducing microservices?

THE 30-SECOND ANSWER

Do not split a small or rapidly changing product merely to gain more services. A modular monolith is often simpler until a real boundary is clear.

CONCEPT → REAL EXAMPLE → RISK

Prefer a modular monolith when domain boundaries are unclear, the team is...	→	A five-person team building a changing product keeps clean modules and one... → Premature boundaries can turn every feature into cross-team work

EXAMPLE FROM THE ORIGINAL GUIDE

A five-person team building a changing product keeps clean modules and one deployment. Later, a stable fraud capability with distinct specialists becomes a candidate for extraction.

INTERVIEWER'S FOLLOW-UP

What extraction trigger would you put in an ADR?

Answer → Write an ADR trigger such as repeated release blocking, independent scaling demand or stable domain ownership; name the metric and review date.

COMMON INTERVIEW TRAP: Premature boundaries can turn every feature into cross-team work.

MY ONE-LINE RECAP _____

CHAPTER 02/24

Domain boundaries and ownership

Use the domain language to place ownership boundaries.

Five interview questions

Q006. What is a bounded context in domain-driven design?

Q007. What is an aggregate, and why does it matter?

Q008. How do you decide the right service size?

Q009. How should service and data ownership work?

Q010. Which code should services share?

Your challenge

Explain one correct operation, one failure path and the evidence that would tell you the design is working.

Domain boundaries and ownership

Q006. What is a bounded context in domain-driven design?

THE 30-SECOND ANSWER

A bounded context is a place where a business word has one meaning. Billing and CRM may describe a customer differently and should not share internal entities.

CONCEPT → REAL EXAMPLE → RISK

A bounded context defines where a domain model and its language have a… → Customer in CRM means a relationship and contact history; customer in billing… → An identical noun does not imply an identical model

EXAMPLE FROM THE ORIGINAL GUIDE

> Customer in CRM means a relationship and contact history; customer in billing means invoice identity and payment terms. Translate between those models instead of sharing one giant Customer entity.

INTERVIEWER'S FOLLOW-UP

How would the two contexts exchange customer changes?

Answer → Publish an agreed CustomerChanged event or a versioned query API. Translate into each context's local model and handle duplicates and delayed delivery.

COMMON INTERVIEW TRAP: An identical noun does not imply an identical model.

MY ONE-LINE RECAP _____

Domain boundaries and ownership

Q007. What is an aggregate, and why does it matter?

THE 30-SECOND ANSWER

An aggregate groups data that must follow one business rule in one local transaction. Its root checks rules for all members.

CONCEPT → REAL EXAMPLE → RISK

An aggregate is a consistency boundary containing entities and value... → An Order root rejects a negative quantity and owns its line items → An aggregate should not become the entire database graph

EXAMPLE FROM THE ORIGINAL GUIDE

An Order root rejects a negative quantity and owns its line items. Inventory availability belongs to a separate owner and is represented by a reservation result, not an Order-side update of stock tables.

INTERVIEWER'S FOLLOW-UP

Which rule must be atomic, and which can complete later?

Answer → Order totals and line-item quantities should agree atomically. Email notifications or search indexing can follow later through reliable events.

COMMON INTERVIEW TRAP: An aggregate should not become the entire database graph.

MY ONE-LINE RECAP _____

Domain boundaries and ownership

Q008. How do you decide the right service size?

THE 30-SECOND ANSWER

Size services by business changes, data ownership and operational needs, not file counts. Too-small boundaries create lots of slow calls.

CONCEPT → REAL EXAMPLE → RISK

Group behavior that changes together and requires closely related invariants → Returns and refund policy may belong together initially → Excessive synchronous chatter can reveal a misplaced boundary

EXAMPLE FROM THE ORIGINAL GUIDE

Returns and refund policy may belong together initially. If every return requires ten remote calls to read tiny fragments, reconsider the split before adding more orchestration.

IMPLEMENTATION SKETCH *illustrative*

```
// Keep a single owner of stock semantics:
Reservation result = inventory.reserve(command);
```

INTERVIEWER'S FOLLOW-UP

Which signals would justify merging two services?

Answer → Merge if changes repeatedly touch both services, most requests require synchronous round trips, or ownership and data invariants cannot be separated economically.

COMMON INTERVIEW TRAP: Excessive synchronous chatter can reveal a misplaced boundary.

MY ONE-LINE RECAP _____

Domain boundaries and ownership

Q009. How should service and data ownership work?

THE 30-SECOND ANSWER

A service team should own its APIs, data rules, operations, monitoring and releases. Other teams use its contract rather than editing its tables.

CONCEPT → REAL EXAMPLE → RISK

Assign a team accountable for the contract, data lifecycle, reliability and... → The inventory team owns reservation semantics and schema migrations → Ownership without operational responsibility leaves failures unowned

EXAMPLE FROM THE ORIGINAL GUIDE

> The inventory team owns reservation semantics and schema migrations. Checkout requests a reservation and handles rejection; it does not insert directly into inventory tables.

INTERVIEWER'S FOLLOW-UP

Who approves a breaking contract change?

Answer → The API owner proposes the version and migration plan; consumer owners approve or verify compatibility through contract tests and a deprecation window.

COMMON INTERVIEW TRAP: Ownership without operational responsibility leaves failures unowned.

MY ONE-LINE RECAP _____

Domain boundaries and ownership

Q010. Which code should services share?

THE 30-SECOND ANSWER

Share stable technical helpers cautiously. Avoid common domain objects that force teams to upgrade in sync.

CONCEPT → REAL EXAMPLE → RISK

Share small, stable technical components such as telemetry helpers or... → Each service uses a logging starter, but billing and CRM keep separate customer... → A shared DTO can turn an internal refactor into many deployments

EXAMPLE FROM THE ORIGINAL GUIDE

Each service uses a logging starter, but billing and CRM keep separate customer DTOs. Events expose stable business facts, not serialized persistence entities from a shared jar.

INTERVIEWER'S FOLLOW-UP

How would you evolve a shared starter safely?

Answer → Publish a versioned starter, follow semantic compatibility, test it against supported Boot versions and allow a staged upgrade with old versions working meanwhile.

COMMON INTERVIEW TRAP: A shared DTO can turn an internal refactor into many deployments.

MY ONE-LINE RECAP _____

CHAPTER 03/24

REST contracts and API evolution

Contracts must survive old and new clients running together.

Five interview questions

Q011. What do HTTP methods and status codes mean in a service API?

Q012. How do you make a create or payment API idempotent?

Q013. How should a microservice return errors?

Q014. How do pagination and sorting affect API correctness?

Q015. How can services evolve APIs independently?

Your challenge

Explain one correct operation, one failure path and the evidence that would tell you the design is working.

REST contracts and API evolution

Q011. What do HTTP methods and status codes mean in a service API?

THE 30-SECOND ANSWER

GET reads; POST commonly creates or commands; PUT replaces the addressed resource; DELETE removes it. Status codes tell clients what actually happened.

CONCEPT → REAL EXAMPLE → RISK

Use methods according to their semantics: GET reads, POST performs...	→	POST /orders returns 201 with Location when an order is created; 202 means work...
→		GET-triggered charging is unsafe even if it appears convenient

EXAMPLE FROM THE ORIGINAL GUIDE

POST /orders returns 201 with Location when an order is created; 202 means work was accepted for later processing, not that payment succeeded. A repeated DELETE can return 404 after an earlier success while remaining idempotent.

IMPLEMENTATION SKETCH *illustrative*

```
POST /orders    -> 201 Created + Location
POST /jobs      -> 202 Accepted + operationId
```

INTERVIEWER'S FOLLOW-UP

When would a create operation return 202?

Answer → Return HTTP 202 when the request has been durably accepted but processing is still pending. Supply an operation ID and status endpoint; do not imply success.

COMMON INTERVIEW TRAP: GET-triggered charging is unsafe even if it appears convenient.

MY ONE-LINE RECAP _____

REST contracts and API evolution

Q012. How do you make a create or payment API idempotent?

THE 30-SECOND ANSWER

Give each create or charge a stable idempotency key scoped to user/operation. Store its request fingerprint and result durably so retries cannot repeat the effect.

CONCEPT → REAL EXAMPLE → RISK

Define a client-supplied operation key scoped to the authenticated tenant... → Tenant A retries payment key p-17 after a timeout → An in-memory map does not coordinate replicas or survive restart

EXAMPLE FROM THE ORIGINAL GUIDE

Tenant A retries payment key p-17 after a timeout. The server returns the original payment reference. Tenant B's same key is a different scope; changing A's amount under p-17 produces a conflict.

IMPLEMENTATION SKETCH *illustrative*

```
// Key identity includes tenant and operation:
record Key(String tenant, String action, String id) {}
```

INTERVIEWER'S FOLLOW-UP

How do key expiry and unknown external outcomes interact?

Answer → Keep keys at least as long as the replay/recovery period. Before expiry or reuse, resolve unknown external charges by original operation identity, not a fresh payment.

COMMON INTERVIEW TRAP: An in-memory map does not coordinate replicas or survive restart.

MY ONE-LINE RECAP _____

REST contracts and API evolution

Q013. How should a microservice return errors?

THE 30-SECOND ANSWER

Return a documented status and machine-readable error code. Clients need to distinguish bad input, denied access, business rejection and temporary failure.

CONCEPT → REAL EXAMPLE → RISK

Provide a stable machine-readable contract with a documented code or... → A reservation rejection returns a documented stock-unavailable problem → Clients should not parse human-readable error messages

EXAMPLE FROM THE ORIGINAL GUIDE

A reservation rejection returns a documented stock-unavailable problem. A database connection failure returns a generic service error and trace reference, not SQL text, credentials or a stack trace.

IMPLEMENTATION SKETCH *illustrative*

```
// RFC 9457-style problem details:
{ "type":"/problems/out-of-stock", "status":409 }
```

INTERVIEWER'S FOLLOW-UP

Which errors may a caller safely retry?

Answer → Retry transient 429/503/network errors only within a deadline and when the operation is safe to repeat. Do not blindly retry validation or permission failures.

COMMON INTERVIEW TRAP: Clients should not parse human-readable error messages.

MY ONE-LINE RECAP _____

REST contracts and API evolution

Q014. How do pagination and sorting affect API correctness?

THE 30-SECOND ANSWER

Sorting needs a unique tie-breaker. Keyset/cursor pagination avoids many offset problems but changing records can still move between pages.

CONCEPT → REAL EXAMPLE → RISK

Specify a stable ordering and a unique tie-breaker → For newest orders, sort by created_at descending and id descending → A cursor is not automatically a consistent snapshot

EXAMPLE FROM THE ORIGINAL GUIDE

For newest orders, sort by created_at descending and id descending. A cursor contains both values; using only the timestamp can skip or repeat orders created at the same instant.

IMPLEMENTATION SKETCH *illustrative*

```
ORDER BY created_at DESC, order_id DESC
-- Cursor includes BOTH sort columns.
```

INTERVIEWER'S FOLLOW-UP

How would you handle changes during a long export?

Answer → For a consistent export, use an owner-supported snapshot or a fixed watermark, carry stable cursor/version information and reconcile changes after the export.

COMMON INTERVIEW TRAP: A cursor is not automatically a consistent snapshot.

MY ONE-LINE RECAP _____

REST contracts and API evolution

Q015. How can services evolve APIs independently?

THE 30-SECOND ANSWER

Make additive, compatible API changes first. Give old clients time to upgrade and test both old and new combinations.

CONCEPT → REAL EXAMPLE → RISK

Prefer compatible additive changes, tolerant readers and explicit… → Add optional deliveryInstructions while preserving existing defaults → Syntactically valid JSON can still be semantically incompatible

EXAMPLE FROM THE ORIGINAL GUIDE

> Add optional deliveryInstructions while preserving existing defaults. A move from one total to multiple tax components needs a migration contract rather than silently changing what total means.

INTERVIEWER'S FOLLOW-UP

What evidence is required before retiring an API version?

Answer → Require measured old-version traffic near zero, verified consumer upgrades, passing compatibility tests, a communicated retirement window and rollback instructions.

COMMON INTERVIEW TRAP: Syntactically valid JSON can still be semantically incompatible.

MY ONE-LINE RECAP _____

CHAPTER 04/24

Communication and Java clients

Synchronous and asynchronous work both need failure rules.

Five interview questions

Q016. When should services communicate synchronously or asynchronously?

Q017. How do REST and gRPC differ for microservices?

Q018. How do RestClient, WebClient, HTTP Service Clients and Feign differ?

Q019. What coupling remains with asynchronous events?

Q020. How do you design an aggregation endpoint?

Your challenge

Explain one correct operation, one failure path and the evidence that would tell you the design is working.

Communication and Java clients

Q016. When should services communicate synchronously or asynchronously?

THE 30-SECOND ANSWER

Call another service synchronously when the answer is needed now. Use a message when the work can safely finish later and the user can see a pending state.

CONCEPT → REAL EXAMPLE → RISK

Use a synchronous call when the caller needs an immediate result and can... → Checkout may synchronously validate an address, then publish OrderAccepted for... → A broker moves work into a queue; it does not eliminate failure

EXAMPLE FROM THE ORIGINAL GUIDE

Checkout may synchronously validate an address, then publish OrderAccepted for notification. Email failure should not prevent an accepted order from being stored.

INTERVIEWER'S FOLLOW-UP

How does a user learn the final asynchronous result?

Answer → Return 202 and an operation ID; offer GET status, webhooks or notifications. Keep final state at the owning service and make retries idempotent.

COMMON INTERVIEW TRAP: A broker moves work into a queue; it does not eliminate failure.

MY ONE-LINE RECAP _____

Communication and Java clients

Q017. How do REST and gRPC differ for microservices?

THE 30-SECOND ANSWER

REST often uses human-readable HTTP/JSON contracts. gRPC uses Protobuf methods and supports efficient streaming, but brings tooling and compatibility needs.

CONCEPT → REAL EXAMPLE → RISK

REST commonly uses HTTP resource semantics and interoperable... → A public order API uses JSON over HTTP → A faster transport does not remove downstream database latency

EXAMPLE FROM THE ORIGINAL GUIDE

A public order API uses JSON over HTTP. An internal high-volume telemetry stream may use gRPC if clients, proxies and observability support its transport and contracts.

IMPLEMENTATION SKETCH *illustrative*

```
// Protobuf: never reuse a removed field number.
message Order { string id = 1; }
```

INTERVIEWER'S FOLLOW-UP

How would you evolve a protobuf message without breaking clients?

Answer → Do not reuse field numbers or change their meanings. Add optional fields, reserve removed numbers/names and test older and newer generated consumers.

COMMON INTERVIEW TRAP: A faster transport does not remove downstream database latency.

MY ONE-LINE RECAP _____

Communication and Java clients

Q018. How do RestClient, WebClient, HTTP Service Clients and Feign differ?

THE 30-SECOND ANSWER

RestClient is synchronous; WebClient supports reactive calls; HTTP interfaces describe clients declaratively; Feign uses Spring Cloud integration.

CONCEPT → REAL EXAMPLE → RISK

RestClient offers a synchronous fluent API; WebClient offers reactive... → A servlet application can use RestClient behind an InventoryPort → Framework generation and starter compatibility matter; verify the selected...

EXAMPLE FROM THE ORIGINAL GUIDE

A servlet application can use RestClient behind an InventoryPort. A reactive handler uses WebClient without blocking its event loop. Either can expose a declarative contract through an HTTP service interface.

IMPLEMENTATION SKETCH *illustrative*

```
@HttpExchange("/inventory")
interface InventoryApi {
  @GetExchange("/{id}") Inventory get(@PathVariable String id);
}
```

INTERVIEWER'S FOLLOW-UP

Where would you configure transport timeouts and pool limits?

Answer → Set connect, read/response and pool-acquisition timeouts at the underlying client, plus bounded connection-pool and total-request deadlines. Test against actual client version.

COMMON INTERVIEW TRAP: Framework generation and starter compatibility matter; verify the selected release.

MY ONE-LINE RECAP _____

Communication and Java clients

Q019. What coupling remains with asynchronous events?

THE 30-SECOND ANSWER

Messages remove immediate waiting but consumers still depend on event identity, meaning, schema and ordering. An event is a long-lived contract.

CONCEPT → REAL EXAMPLE → RISK

Consumers still depend on event meaning, schema, identity, partition keys,... → InventoryReserved means a committed reservation exists → Asynchronous does not mean loosely coupled by definition

EXAMPLE FROM THE ORIGINAL GUIDE

InventoryReserved means a committed reservation exists. Renaming it to mean only that a reservation was requested changes a business fact and breaks order progression, even if the JSON shape is unchanged.

INTERVIEWER'S FOLLOW-UP

How would you retire an event field?

Answer → Keep the old field during migration, add the replacement, watch real consumer adoption and remove only after all replay and consumer compatibility windows close.

COMMON INTERVIEW TRAP: Asynchronous does not mean loosely coupled by definition.

MY ONE-LINE RECAP _____

Communication and Java clients

Q020. How do you design an aggregation endpoint?

THE 30-SECOND ANSWER

An aggregation endpoint combines several services. Bound parallel calls and clearly mark missing optional sections instead of hiding failures.

CONCEPT → REAL EXAMPLE → RISK

Call independent dependencies in parallel only when the resource budget... → A product page requires product identity, but reviews are optional → Parallel calls can increase load and exhaust pools faster

EXAMPLE FROM THE ORIGINAL GUIDE

A product page requires product identity, but reviews are optional. A review timeout produces a labelled unavailable section while purchase eligibility still comes from the authoritative owner.

INTERVIEWER'S FOLLOW-UP

How would you stop optional work after the response deadline?

Answer → Carry one deadline through child calls, cancel supported local futures or subscriptions when the response expires and release permits; do not assume remote effects were undone.

COMMON INTERVIEW TRAP: Parallel calls can increase load and exhaust pools faster.

MY ONE-LINE RECAP _____

CHAPTER 05/24

Discovery, gateways and service mesh

Routing, discovery and permission are separate decisions.

Five interview questions

Q021. How does service discovery work?

Q022. What is client-side versus server-side load balancing?

Q023. What belongs in an API gateway?

Q024. What is a backend for frontend?

Q025. What does a service mesh provide, and what does it not provide?

Your challenge

Explain one correct operation, one failure path and the evidence that would tell you the design is working.

Discovery, gateways and service mesh

Q021. How does service discovery work?

THE 30-SECOND ANSWER

Discovery turns a service name into active endpoints. Membership can change, so a cached address may become stale.

CONCEPT → REAL EXAMPLE → RISK

A caller resolves a logical service name to reachable endpoints through... → In Kubernetes, order calls inventory through a Service DNS name → Do not add two discovery systems without a concrete need

EXAMPLE FROM THE ORIGINAL GUIDE

In Kubernetes, order calls inventory through a Service DNS name. Outside that platform, an application registry such as Eureka can supply inventory instances to a discovery-aware client.

IMPLEMENTATION SKETCH *illustrative*

```
// Service discovery example (Kubernetes DNS):
http://inventory.default.svc.cluster.local
```

INTERVIEWER'S FOLLOW-UP

What happens when a cached endpoint has disappeared?

Answer → Use a bounded connection failure, refresh endpoints and retry only safe calls using another healthy instance. Keep request deadlines and avoid infinite rediscovery loops.

COMMON INTERVIEW TRAP: Do not add two discovery systems without a concrete need.

MY ONE-LINE RECAP _____

Discovery, gateways and service mesh

Q022. What is client-side versus server-side load balancing?

THE 30-SECOND ANSWER

With client-side balancing the caller picks an instance; with server-side balancing a proxy or platform picks one.

CONCEPT → REAL EXAMPLE → RISK

Client-side balancing selects an instance before making the call → A Spring Cloud LoadBalancer-enabled client resolves instances itself → A service name alone does not activate a discovery-aware Java client

EXAMPLE FROM THE ORIGINAL GUIDE

A Spring Cloud LoadBalancer-enabled client resolves instances itself. A caller using a Kubernetes Service address relies on platform routing; a persistent connection may continue to one selected backend.

INTERVIEWER'S FOLLOW-UP

How would you detect uneven traffic across replicas?

Answer → Compare per-replica request counts, latency, active connections, errors and target selection logs; check keep-alive connections and uneven traffic keys.

COMMON INTERVIEW TRAP: A service name alone does not activate a discovery-aware Java client.

MY ONE-LINE RECAP _____

Discovery, gateways and service mesh

Q023. What belongs in an API gateway?

THE 30-SECOND ANSWER

A gateway routes and applies shared HTTP policies, but the service still owns business permission and correct data changes.

CONCEPT → REAL EXAMPLE → RISK

A gateway provides client-facing routing and selected cross-cutting… → The gateway routes /orders to the order service and enforces a payload limit → Gateway authentication does not replace object-level authorization

EXAMPLE FROM THE ORIGINAL GUIDE

The gateway routes /orders to the order service and enforces a payload limit. The order service checks whether the caller may cancel that particular order.

INTERVIEWER'S FOLLOW-UP

Which policies must also run inside a service?

Answer → Repeat object-level tenant authorization, input validation, business idempotency and data ownership checks within the service. Never rely on gateway identity alone.

COMMON INTERVIEW TRAP: Gateway authentication does not replace object-level authorization.

MY ONE-LINE RECAP _____

Discovery, gateways and service mesh

Q024. What is a backend for frontend?

THE 30-SECOND ANSWER

A BFF shapes responses for one type of client, such as mobile, while the business rules stay in domain services.

CONCEPT → REAL EXAMPLE → RISK

A BFF exposes a client-specific API, reducing excessive round trips and... → A mobile BFF returns a compact dashboard with five recent orders → A BFF should not become the only owner of core business rules

EXAMPLE FROM THE ORIGINAL GUIDE

A mobile BFF returns a compact dashboard with five recent orders. A web BFF returns richer filtering. Both use the same order cancellation capability and ownership checks.

INTERVIEWER'S FOLLOW-UP

When would a single gateway be enough?

Answer → A single gateway is enough when clients need similar payloads and little client-specific orchestration, so separate BFF deployments would only add complexity.

COMMON INTERVIEW TRAP: A BFF should not become the only owner of core business rules.

MY ONE-LINE RECAP _____

Discovery, gateways and service mesh

Q025. What does a service mesh provide, and what does it not provide?

THE 30-SECOND ANSWER

A mesh can handle service transport, identity and some routing. It cannot decide whether an order may be charged or how a retry avoids duplicates.

CONCEPT → REAL EXAMPLE → RISK

A mesh can move transport concerns such as workload-to-workload routing,... → A mesh authenticates workload identities and encrypts order-to-payment traffic → Mesh and application retries can multiply if both are enabled

EXAMPLE FROM THE ORIGINAL GUIDE

A mesh authenticates workload identities and encrypts order-to-payment traffic. Payment still validates operation scope, tenant ownership and idempotency for a charge request.

INTERVIEWER'S FOLLOW-UP

How would you assign retry ownership between the two layers?

Answer → Choose one retry owner per call. Disable overlapping mesh/app attempts; the application owns business-safe retry decisions and the mesh may manage narrow transport failures.

COMMON INTERVIEW TRAP: Mesh and application retries can multiply if both are enabled.

MY ONE-LINE RECAP _____

CHAPTER 06/24

Spring Boot implementation and configuration

Choose compatible framework versions and keep mutable data safe.

Five interview questions

Q026. What makes a Spring Boot application a microservice?

Q027. How should Spring Boot and Spring Cloud versions be selected?

Q028. How should configuration and secrets be handled?

Q029. Are Spring singleton beans safe for concurrent requests?

Q030. How should Spring health endpoints be used?

Your challenge

Explain one correct operation, one failure path and the evidence that would tell you the design is working.

Spring Boot implementation and configuration

Q026. What makes a Spring Boot application a microservice?

THE 30-SECOND ANSWER

Spring Boot provides the application framework. It becomes a microservice because it owns a real business capability and private state, not because of an annotation.

CONCEPT → REAL EXAMPLE → RISK

Boot assembles an application with dependency injection, web support,...	An InventoryApplication has an HTTP adapter, reservation use case and inventory...	Start with behavior and invariants, then select infrastructure

EXAMPLE FROM THE ORIGINAL GUIDE

An InventoryApplication has an HTTP adapter, reservation use case and inventory repository. Adding @SpringBootApplication to five applications without ownership boundaries does not create five independent capabilities.

IMPLEMENTATION SKETCH *illustrative*

```
@RestController class OrderController {
  private final OrderUseCase useCase;
  // Keep business rules behind a domain port.
}
```

INTERVIEWER'S FOLLOW-UP

How would you structure ports and adapters inside the service?

Answer → Put HTTP controllers and repositories in adapters, use cases in the application layer, and business rules in the domain. Depend inward through ports/interfaces.

COMMON INTERVIEW TRAP: Start with behavior and invariants, then select infrastructure.

MY ONE-LINE RECAP _____

Spring Boot implementation and configuration

Q027. How should Spring Boot and Spring Cloud versions be selected?

THE 30-SECOND ANSWER

Choose one tested Spring Boot and Spring Cloud combination with managed dependency versions. Newest individual starters may not work together.

CONCEPT → REAL EXAMPLE → RISK

Choose an explicitly compatible Boot and Cloud release train and use the... → Generate a project from Spring Initializr for a chosen Boot release and selected... → This guide is not a claim that every shown API works on every Boot release

EXAMPLE FROM THE ORIGINAL GUIDE

> Generate a project from Spring Initializr for a chosen Boot release and selected Cloud features. Keep the generated Cloud dependency-management import instead of independently pinning every starter.

INTERVIEWER'S FOLLOW-UP

How would you investigate a NoSuchMethodError after an upgrade?

Answer → Inspect the first NoSuchMethodError stack, identify runtime jar versions, inspect dependency tree/BOM overrides and reproduce with a clean supported release set.

COMMON INTERVIEW TRAP: This guide is not a claim that every shown API works on every Boot release.

MY ONE-LINE RECAP _____

Spring Boot implementation and configuration

Q028. How should configuration and secrets be handled?

THE 30-SECOND ANSWER

Keep environment settings outside code, validate them, and load secrets through approved providers. A secret update must reach the live connections safely.

CONCEPT → REAL EXAMPLE → RISK

Separate application configuration from deployable code, validate required... → Inventory binds maximumReservationMinutes through validated configuration... → Do not assume refreshing a property updates every existing connection

EXAMPLE FROM THE ORIGINAL GUIDE

Inventory binds maximumReservationMinutes through validated configuration properties. A database password is provided through a protected runtime source rather than committed in application-prod.yml.

IMPLEMENTATION SKETCH *illustrative*

```
@ConfigurationProperties(prefix="reservation")
record Limits(int maxMinutes) {}
```

INTERVIEWER'S FOLLOW-UP

What happens during secret rotation with active pools?

Answer → New credentials must overlap long enough for old pools to drain. Refresh or recreate connections, verify authentication, then revoke the old credential.

COMMON INTERVIEW TRAP: Do not assume refreshing a property updates every existing connection.

MY ONE-LINE RECAP _____

Spring Boot implementation and configuration

Q029. Are Spring singleton beans safe for concurrent requests?

THE 30-SECOND ANSWER

A Spring singleton is shared by many request threads; its fields are not magically thread-safe. Shared business writes need database-level protection across replicas.

CONCEPT → REAL EXAMPLE → RISK

Singleton defines sharing scope, not thread safety → A controller field currentCustomer leaks state between concurrent requests → Stateless HTTP handlers may still use shared stateful dependencies

EXAMPLE FROM THE ORIGINAL GUIDE

A controller field currentCustomer leaks state between concurrent requests. A local synchronized block around stock updates also fails when another replica writes the same stock row.

IMPLEMENTATION SKETCH *illustrative*

```
// One pod lock does not protect other pods.
UPDATE stock SET qty = qty - 1
 WHERE sku = :sku AND qty >= 1;
```

INTERVIEWER'S FOLLOW-UP

Where should a cross-replica stock invariant be enforced?

Answer → Use one atomic conditional stock update or transaction at the Inventory owner's database, not a synchronized Java method on one pod.

COMMON INTERVIEW TRAP: Stateless HTTP handlers may still use shared stateful dependencies.

MY ONE-LINE RECAP _____

Spring Boot implementation and configuration

Q030. How should Spring health endpoints be used?

THE 30-SECOND ANSWER

Liveness asks whether restarting might help; readiness asks whether to send traffic; startup gives initial work time. Keep probes safe and meaningful.

CONCEPT → REAL EXAMPLE → RISK

Expose carefully selected operational endpoints and keep sensitive...

→

A recommendation dependency outage can produce degraded product pages while the...

→

Putting every downstream service into liveness can cause restart storms

EXAMPLE FROM THE ORIGINAL GUIDE

A recommendation dependency outage can produce degraded product pages while the API stays ready. A required local schema initialization failure keeps the service unready until it can serve its contract.

IMPLEMENTATION SKETCH *illustrative*

```
management.endpoint.health.probes.enabled=true
// Separate traffic readiness from restart health.
```

INTERVIEWER'S FOLLOW-UP

Which dependencies belong in readiness for this service?

Answer → Include only dependencies whose absence makes every request invalid or unsafe. Optional recommendations should degrade; transient payment outage rarely justifies restarting the service.

COMMON INTERVIEW TRAP: Putting every downstream service into liveness can cause restart storms.

MY ONE-LINE RECAP _____

PART II

FAILURE, CONSISTENCY & WORKFLOWS

Unknown outcomes, durable state and one real authority.

Question 031-060

CHAPTER 07/24

Timeouts, deadlines and retries

The clock can expire without undoing a remote effect.

Five interview questions

Q031. Does a timeout mean the remote operation failed?

Q032. How do connect, pool-acquisition and response timeouts differ?

Q033. Which failures should be retried?

Q034. Why use backoff and jitter, and what is retry amplification?

Q035. Does cancelling a Future stop remote work?

Your challenge

Explain one correct operation, one failure path and the evidence that would tell you the design is working.

Timeouts, deadlines and retries

Q031. Does a timeout mean the remote operation failed?

THE 30-SECOND ANSWER

A timeout means the answer did not arrive in time. The payment may have succeeded, so do not charge again with a new key.

CONCEPT → REAL EXAMPLE → RISK

No	→ Payment commits charge p-17 at 950 ms; order times out at 900 ms	→ A 504 response is not proof that no business effect occurred

EXAMPLE FROM THE ORIGINAL GUIDE

Payment commits charge p-17 at 950 ms; order times out at 900 ms. Retrying with a new key can charge twice. Reuse the operation identity and ask for its recorded outcome.

IMPLEMENTATION SKETCH *illustrative*

```
POST /payments
Idempotency-Key: pay-17
// On timeout: GET /payments/operations/pay-17
```

INTERVIEWER'S FOLLOW-UP

Which state should the order show while the outcome is unknown?

Answer → Show PAYMENT_PENDING or PAYMENT_UNKNOWN, save the operation ID and reconcile with the payment owner before confirming or cancelling.

COMMON INTERVIEW TRAP: A 504 response is not proof that no business effect occurred.

MY ONE-LINE RECAP _____

Timeouts, deadlines and retries

Q032. How do connect, pool-acquisition and response timeouts differ?

THE 30-SECOND ANSWER

Connect timeout covers opening a connection; pool timeout covers waiting for one; response timeout covers remote response behavior. Total deadline includes all waiting.

CONCEPT → REAL EXAMPLE → RISK

Connect timeout bounds connection establishment → An order request has a 1,000 ms deadline → Socket read timeouts may reset on progress rather than cap total elapsed time

EXAMPLE FROM THE ORIGINAL GUIDE

An order request has a 1,000 ms deadline. It spends 200 ms in a queue, leaving only 800 ms for application work and remote calls, even if a client's response timeout is configured as 1,000 ms.

IMPLEMENTATION SKETCH *illustrative*

```
long remaining = deadlineNanos - System.nanoTime();
if (remaining <= 0) return expired();
```

INTERVIEWER'S FOLLOW-UP

How would you propagate the remaining deadline?

Answer → Send the remaining end-to-end budget with a request header or context, recalculate before each call and skip attempts that cannot finish in time.

COMMON INTERVIEW TRAP: Socket read timeouts may reset on progress rather than cap total elapsed time.

MY ONE-LINE RECAP _____

Timeouts, deadlines and retries

Q033. Which failures should be retried?

THE 30-SECOND ANSWER

Retry only likely temporary failures and only when repeating the business action is safe. Limit attempts, time and overall retry pressure.

CONCEPT → REAL EXAMPLE → RISK

Retry classified transient failures only when the operation is idempotent... → A connection failure during an idempotent inventory-status read may be retried → An exception whitelist alone does not establish business retry safety

EXAMPLE FROM THE ORIGINAL GUIDE

A connection failure during an idempotent inventory-status read may be retried. Insufficient stock is a business result. A payment timeout requires the same operation key and an outcome-recovery contract.

INTERVIEWER'S FOLLOW-UP

Who owns retries when several layers can repeat a call?

Answer → Make the application boundary responsible for its retry budget and disable other layers or allow only explicitly coordinated low-level retries.

COMMON INTERVIEW TRAP: An exception whitelist alone does not establish business retry safety.

MY ONE-LINE RECAP _____

Timeouts, deadlines and retries

Q034. Why use backoff and jitter, and what is retry amplification?

THE 30-SECOND ANSWER

Backoff waits between retries; jitter avoids many clients retrying together. Retrying at multiple layers multiplies traffic during an outage.

CONCEPT → REAL EXAMPLE → RISK

Backoff spaces attempts; jitter spreads synchronized clients → Three layers making three total attempts each can produce up to 27 deepest calls… → Say whether an attempt count includes the original call

EXAMPLE FROM THE ORIGINAL GUIDE

Three layers making three total attempts each can produce up to 27 deepest calls for one logical request, rather than three. Adding random delay changes timing but not that worst-case count.

IMPLEMENTATION SKETCH *illustrative*

```
// Example: bounded retries with jitter
delayMs = random.nextLong(maxDelayMs + 1);
```

INTERVIEWER'S FOLLOW-UP

How would you cap retries during a regional outage?

Answer → Set one small budget, cap attempts and total elapsed time, use circuit breakers and reject overload. During regional loss, fail fast or move traffic safely.

COMMON INTERVIEW TRAP: Say whether an attempt count includes the original call.

MY ONE-LINE RECAP _____

Timeouts, deadlines and retries

Q035. Does cancelling a Future stop remote work?

THE 30-SECOND ANSWER

Cancelling a Future may stop local waiting but cannot undo a remote transaction that already succeeded.

CONCEPT → REAL EXAMPLE → RISK

Cancellation and interruption are requests whose effects depend on the... → An optional recommendation call outlives its parent response after orTimeout → Interruption should not be swallowed and converted into an ordinary retry

EXAMPLE FROM THE ORIGINAL GUIDE

An optional recommendation call outlives its parent response after orTimeout. Release local resources where possible and ensure the remaining work cannot create untracked business effects.

INTERVIEWER'S FOLLOW-UP

How do you verify cancellation and pool release behavior?

Answer → Test a slow dependency with connection/pool metrics: cancel a request, confirm permits and sockets are released, and separately verify the remote outcome.

COMMON INTERVIEW TRAP: Interruption should not be swallowed and converted into an ordinary retry.

MY ONE-LINE RECAP _____

CHAPTER 08/24

Resilience and admission control

Bound the resources before overload spreads.

Five interview questions

Q036. How does a circuit breaker work?

Q037. What is the bulkhead pattern?

Q038. How do rate limiting and concurrency limiting differ?

Q039. What makes a fallback safe?

Q040. What is backpressure, and how is it different from a queue?

Your challenge

Explain one correct operation, one failure path and the evidence that would tell you the design is working.

Resilience and admission control

Q036. How does a circuit breaker work?

THE 30-SECOND ANSWER

A circuit breaker stops repeated calls after enough failures, then tests recovery with limited probes. It is not a bulkhead or timeout.

CONCEPT → REAL EXAMPLE → RISK

A normal circuit breaker observes recorded outcomes while closed, rejects... → After enough recorded inventory failures, the breaker opens → A breaker is not a limit on all concurrent calls while closed

EXAMPLE FROM THE ORIGINAL GUIDE

After enough recorded inventory failures, the breaker opens. Order fails fast or returns a documented pending response. Half-open probes test recovery before normal traffic resumes.

IMPLEMENTATION SKETCH *illustrative*

```
// A breaker is not a concurrency cap.
// Use timeouts and a bulkhead separately.
```

INTERVIEWER'S FOLLOW-UP

Which exceptions should count as dependency failures?

Answer → Count transport errors, timeouts and server failures that indicate dependency health. Usually exclude caller validation errors and expected business rejections.

COMMON INTERVIEW TRAP: A breaker is not a limit on all concurrent calls while closed.

MY ONE-LINE RECAP _____

Resilience and admission control

Q037. What is the bulkhead pattern?

THE 30-SECOND ANSWER

A bulkhead reserves independent capacity for different downstream calls. A slow optional service should not use up checkout's entire worker budget.

CONCEPT → REAL EXAMPLE → RISK

A bulkhead isolates a resource budget, commonly concurrent calls, worker... → Recommendation requests get a small semaphore budget; checkout gets a separate... → A permit count on each replica is not a global limit

EXAMPLE FROM THE ORIGINAL GUIDE

Recommendation requests get a small semaphore budget; checkout gets a separate budget. When recommendations slow down, checkout retains capacity and the product page can omit recommendations.

IMPLEMENTATION SKETCH *illustrative*

```
Semaphore checkoutPermits = new Semaphore(20);
// Acquire only for the protected dependency lifetime.
```

INTERVIEWER'S FOLLOW-UP

How would you size the budget from arrival rate and latency?

Answer → Estimate concurrency from arrival rate × time in service, add measured headroom, then load-test p95, rejected requests and dependency capacity.

COMMON INTERVIEW TRAP: A permit count on each replica is not a global limit.

MY ONE-LINE RECAP _____

Resilience and admission control

Q038. How do rate limiting and concurrency limiting differ?

THE 30-SECOND ANSWER

Rate limiting counts work over time; concurrency limiting caps work currently in progress. Slow work can exhaust resources even at a fixed rate.

CONCEPT → REAL EXAMPLE → RISK

Rate limiting controls admission over time, such as requests per second	→	At 100 requests per second, a 50 ms average residence time implies roughly five in… → Rate, burst size and simultaneous work are separate settings

EXAMPLE FROM THE ORIGINAL GUIDE

At 100 requests per second, a 50 ms average residence time implies roughly five in flight; at two seconds it implies roughly 200 in flight under stable conditions.

IMPLEMENTATION SKETCH *illustrative*

```
// Average in-flight (Little's Law):
concurrency = requestsPerSecond * avgSeconds;
```

INTERVIEWER'S FOLLOW-UP

Should a quota fail open or closed if its store is unavailable?

Answer → Fail closed for sensitive money or authorization limits unless a documented emergency policy exists. A low-risk public read may degrade with a bounded local allowance.

COMMON INTERVIEW TRAP: Rate, burst size and simultaneous work are separate settings.

MY ONE-LINE RECAP _____

Resilience and admission control

Q039. What makes a fallback safe?

THE 30-SECOND ANSWER

A safe fallback can use stale optional display data, but cannot invent payment success, customer permission or real inventory.

CONCEPT → REAL EXAMPLE → RISK

A fallback must preserve the caller's essential business contract or... → Show cached product descriptions with an age marker when catalog reads fail → A fallback can silently turn an availability incident into a correctness...

EXAMPLE FROM THE ORIGINAL GUIDE

> Show cached product descriptions with an age marker when catalog reads fail. Reject or defer a purchase if authoritative stock cannot be reserved; do not return true from a reservation fallback.

INTERVIEWER'S FOLLOW-UP

Which responses may legally contain stale data?

Answer → Allow documented read-only catalog/recommendation responses with TTL and freshness labels. Never use stale data to authorize money, stock or access decisions.

COMMON INTERVIEW TRAP: A fallback can silently turn an availability incident into a correctness incident.

MY ONE-LINE RECAP _____

Resilience and admission control

Q040. What is backpressure, and how is it different from a queue?

THE 30-SECOND ANSWER

Backpressure prevents accepting work faster than it can be finished. A queue only holds a bounded amount; an unlimited queue hides the overload.

CONCEPT → REAL EXAMPLE → RISK

Backpressure or admission control prevents producers from creating more... → A notification worker accepts at most a measured backlog, and the API reports... → An asynchronous method is not automatically a durable job

EXAMPLE FROM THE ORIGINAL GUIDE

A notification worker accepts at most a measured backlog, and the API reports overload or durable pending work according to its contract. It does not keep adding tasks to an unlimited executor queue.

IMPLEMENTATION SKETCH *illustrative*

```
BlockingQueue<Job> queue = new ArrayBlockingQueue<>(100);
boolean accepted = queue.offer(job);
```

INTERVIEWER'S FOLLOW-UP

What happens when producers outpace consumers for an hour?

Answer → The backlog and delay grow without bound unless admission slows, producers are throttled, workers scale sustainably or old work is rejected/expired.

COMMON INTERVIEW TRAP: An asynchronous method is not automatically a durable job.

MY ONE-LINE RECAP _____

CHAPTER 09/24

Data ownership and consistency

A private owner, not shared deployment hardware, decides writes.

Five interview questions

Q041. Does database per service require a separate database server?

Q042. Why does @Transactional not create a transaction across services?

Q043. What does the CAP theorem actually say?

Q044. What does PACELC add to the consistency discussion?

Q045. How do you make eventual consistency understandable to users?

Your challenge

Explain one correct operation, one failure path and the evidence that would tell you the design is working.

Data ownership and consistency

Q041. Does database per service require a separate database server?

THE 30-SECOND ANSWER

Database per service means no other service writes its private schema. The services may share a physical database server if access stays separate.

CONCEPT → REAL EXAMPLE → RISK

No	→ Orders and inventory use separate schemas and credentials on one managed cluster	→ Separate credentials protect ownership better than a naming convention alone

EXAMPLE FROM THE ORIGINAL GUIDE

Orders and inventory use separate schemas and credentials on one managed cluster. Their APIs remain independent, while a cluster outage is still a shared infrastructure failure.

INTERVIEWER'S FOLLOW-UP

When would you move a service to a separate cluster?

Answer → Move to a separate cluster when noisy-neighbor load, independent recovery/upgrade needs, security isolation or scaling limits justify the extra operating cost.

COMMON INTERVIEW TRAP: Separate credentials protect ownership better than a naming convention alone.

MY ONE-LINE RECAP _____

Data ownership and consistency

Q042. Why does @Transactional not create a transaction across services?

THE 30-SECOND ANSWER

@Transactional controls a local managed transaction; an HTTP payment is outside that transaction. Remote success remains even if the local order rolls back.

CONCEPT → REAL EXAMPLE → RISK

A Spring transaction applies to resources coordinated by the configured... → Order rolls back its local insert after payment successfully charges through HTTP → Proxy self-invocation and rollback rules also affect local behavior

EXAMPLE FROM THE ORIGINAL GUIDE

Order rolls back its local insert after payment successfully charges through HTTP. The charge remains and must be recovered or compensated through a separate business operation.

IMPLEMENTATION SKETCH *illustrative*

```
@Transactional void saveOrder(Order order) {
  repository.save(order);
  // Remote HTTP payment is NOT part of this DB txn.
}
```

INTERVIEWER'S FOLLOW-UP

Which durable record links a local failure to a successful remote charge?

Answer → Save the external operation ID and a durable workflow state or reconciliation/outbox record connecting the order to that payment outcome.

COMMON INTERVIEW TRAP: Proxy self-invocation and rollback rules also affect local behavior.

MY ONE-LINE RECAP _____

Data ownership and consistency

Q043. What does the CAP theorem actually say?

THE 30-SECOND ANSWER

During a network partition a replicated store cannot promise both strict single-copy consistency and a response from every healthy isolated node.

CONCEPT → REAL EXAMPLE → RISK

For a partitioned distributed store under the theorem's model, it is... → If two disconnected regions both accept conflicting writes to the same balance,... → Practical availability percentages and CAP's formal availability are...

EXAMPLE FROM THE ORIGINAL GUIDE

If two disconnected regions both accept conflicting writes to the same balance, they cannot both guarantee one latest linearizable value. A design must restrict some operations or allow weaker semantics and reconcile.

INTERVIEWER'S FOLLOW-UP

Which operations would you refuse during a partition?

Answer → Refuse conflicting balance, stock and payment writes when their required authority is unreachable. Allow clearly labelled stale catalog reads if the business permits.

COMMON INTERVIEW TRAP: Practical availability percentages and CAP's formal availability are different concepts.

MY ONE-LINE RECAP _____

Data ownership and consistency

Q044. What does PACELC add to the consistency discussion?

THE 30-SECOND ANSWER

PACELC reminds us that consistency can cost latency even without a partition. A closer replica may respond faster but show older data.

CONCEPT → REAL EXAMPLE → RISK

PACELC highlights that tradeoffs also exist without a partition: if there... → Waiting for remote replicas can improve a read or write guarantee while increasing... → The tradeoff depends on replication, protocol and requested operation

EXAMPLE FROM THE ORIGINAL GUIDE

Waiting for remote replicas can improve a read or write guarantee while increasing latency. Serving a local replica can be quicker but expose stale values until replication catches up.

INTERVIEWER'S FOLLOW-UP

Where is stale data acceptable in your user journey?

Answer → Staleness can be acceptable for reviews, recommendations and analytics; not usually for balance checks, payment decisions or stock reservations.

COMMON INTERVIEW TRAP: The tradeoff depends on replication, protocol and requested operation.

MY ONE-LINE RECAP _____

Data ownership and consistency

Q045. How do you make eventual consistency understandable to users?

THE 30-SECOND ANSWER

Users should see PENDING while a delayed update catches up. Define how old a read model may be and how stuck updates get repaired.

CONCEPT → REAL EXAMPLE → RISK

Define the convergence mechanism, acceptable lag and what happens when... → Order creation returns an order ID and PENDING → A lost event without replay or repair can prevent convergence

EXAMPLE FROM THE ORIGINAL GUIDE

Order creation returns an order ID and PENDING. GET /orders/id reads the owner immediately; a search index can take longer and reports its freshness separately.

INTERVIEWER'S FOLLOW-UP

What metric and repair process detect a stuck projection?

Answer → Measure projection lag by event age/version and alert on long gaps. Replay missing events from the durable log or rebuild a projection from owner data.

COMMON INTERVIEW TRAP: A lost event without replay or repair can prevent convergence.

MY ONE-LINE RECAP _____

CHAPTER 10/24

Concurrent writes and data invariants

Protect stock and money where the actual write occurs.

Five interview questions

Q046. What is optimistic locking?

Q047. When is pessimistic locking useful?

Q048. How do you prevent overselling inventory?

Q049. Why are database constraints important even when validation exists?

Q050. What is fencing, and why can a lease lock be insufficient?

Your challenge

Explain one correct operation, one failure path and the evidence that would tell you the design is working.

Concurrent writes and data invariants

Q046. What is optimistic locking?

THE 30-SECOND ANSWER

Optimistic locking compares the version originally read with the current database version. A losing writer sees a conflict instead of silently replacing a change.

CONCEPT → REAL EXAMPLE → RISK

An optimistic update checks the version originally read and advances it... → Two users edit order version 7 → Every write path must respect the version protocol

EXAMPLE FROM THE ORIGINAL GUIDE

Two users edit order version 7. The first update writes version 8. The second update with expected version 7 fails and must reload or return a conflict.

IMPLEMENTATION SKETCH *illustrative*

```
UPDATE orders SET state=:newState, version=version+1
 WHERE id=:id AND version=:expected;
```

INTERVIEWER'S FOLLOW-UP

Would you automatically retry a user-submitted price change?

Answer → Not automatically. A price update reflects a person's decision; reload and ask for confirmation if the value changed, unless business rules explicitly permit recomputation.

COMMON INTERVIEW TRAP: Every write path must respect the version protocol.

MY ONE-LINE RECAP _____

Concurrent writes and data invariants

Q047. When is pessimistic locking useful?

THE 30-SECOND ANSWER

Pessimistic locking holds a database lock during a short local transaction. It can be right for hot rows but increases waiting and deadlock risk.

CONCEPT → REAL EXAMPLE → RISK

Pessimistic locking acquires a database lock before modifying protected... → A short transaction locks the next available settlement row, claims it and commits → A lock inside one database does not lock another service's state

EXAMPLE FROM THE ORIGINAL GUIDE

A short transaction locks the next available settlement row, claims it and commits. It does not hold the lock while calling a slow external bank API.

INTERVIEWER'S FOLLOW-UP

What evidence distinguishes deadlocks from pool exhaustion?

Answer → Database deadlock messages and lock graphs show cycles; pool exhaustion shows connection wait/active counts without necessarily any database lock cycle.

COMMON INTERVIEW TRAP: A lock inside one database does not lock another service's state.

MY ONE-LINE RECAP _____

Concurrent writes and data invariants

Q048. How do you prevent overselling inventory?

THE 30-SECOND ANSWER

Reserve stock with an atomic conditional database update and a reservation ID. A separate stock read followed by update can oversell.

CONCEPT → REAL EXAMPLE → RISK

Enforce the reservation invariant in an atomic operation at the inventory... → UPDATE stock SET available = available - 2 WHERE sku = :sku AND available >= 2 → A local Java synchronized block cannot protect multiple inventory replicas

EXAMPLE FROM THE ORIGINAL GUIDE

UPDATE stock SET available = available - 2 WHERE sku = :sku AND available >= 2. Exactly one affected row indicates a successful decrement; the reservation row and event are written in the same local transaction.

IMPLEMENTATION SKETCH *illustrative*

```
UPDATE inventory SET available=available-:qty
 WHERE sku=:sku AND available>=:qty;
```

INTERVIEWER'S FOLLOW-UP

How do cancellation and reservation expiry avoid double release?

Answer → Use one reservation state transition from ACTIVE to RELEASED or CONFIRMED with a version/unique key. Only the winning transition returns stock; expiry and cancel cannot both release.

COMMON INTERVIEW TRAP: A local Java synchronized block cannot protect multiple inventory replicas.

MY ONE-LINE RECAP _____

Concurrent writes and data invariants

Q049. Why are database constraints important even when validation exists?

THE 30-SECOND ANSWER

Application validation helps users, but database constraints stop races at the real write boundary.

CONCEPT → REAL EXAMPLE → RISK

Application validation provides helpful feedback, but concurrent requests... → Two requests both find no payment for operation key k-5 → A constraint protects only the fields and resources it actually covers

EXAMPLE FROM THE ORIGINAL GUIDE

Two requests both find no payment for operation key k-5. A unique tenant-and-operation constraint prevents both inserts from winning; a transaction records the single resulting payment.

IMPLEMENTATION SKETCH *illustrative*

```
UNIQUE (tenant_id, operation_key)
CHECK (available >= 0)
```

INTERVIEWER'S FOLLOW-UP

Which invariants should remain valid if a second writer bypasses the API?

Answer → Unique idempotency keys, nonnegative stock, valid foreign keys and legal reservation state transitions must remain true even for direct administrative or batch writers.

COMMON INTERVIEW TRAP: A constraint protects only the fields and resources it actually covers.

MY ONE-LINE RECAP _____

Concurrent writes and data invariants

Q050. What is fencing, and why can a lease lock be insufficient?

THE 30-SECOND ANSWER

A lease can expire while a worker is paused. Fencing gives each new owner a higher token that the storage system rejects if it is outdated.

CONCEPT → REAL EXAMPLE → RISK

A process can pause long enough for its lease to expire, then resume... → Worker A owns token 41, pauses, and worker B obtains 42 → Token issuance and resource enforcement require a concrete supported design

EXAMPLE FROM THE ORIGINAL GUIDE

Worker A owns token 41, pauses, and worker B obtains 42. When A resumes, the storage operation rejects 41 after accepting 42 instead of allowing A to overwrite B's work.

IMPLEMENTATION SKETCH *illustrative*

```
// Fencing token must be checked at the real sink:
UPDATE resource SET value=:v, token=:new
 WHERE id=:id AND token < :new;
```

INTERVIEWER'S FOLLOW-UP

Can the external system you call actually enforce a fencing token?

Answer → Only if that system compares tokens atomically at the write boundary. If it cannot, fencing does not protect the effect; use another authoritative conditional-write mechanism.

COMMON INTERVIEW TRAP: Token issuance and resource enforcement require a concrete supported design.

MY ONE-LINE RECAP _____

CHAPTER 11/24

Sagas and workflow recovery

Recover every state, including an unknown payment.

Five interview questions

Q051. What is a saga?

Q052. How do choreography and orchestration differ?

Q053. Is compensation the same as rollback?

Q054. How should a saga handle an unknown payment result?

Q055. How does a workflow survive coordinator crashes?

Your challenge

Explain one correct operation, one failure path and the evidence that would tell you the design is working.

Sagas and workflow recovery

Q051. What is a saga?

THE 30-SECOND ANSWER

A saga is a durable chain of local business transactions. If later work fails, new compensation operations may be needed; it is not one big rollback.

CONCEPT → REAL EXAMPLE → RISK

A saga coordinates a business operation through a sequence of local... → Order acceptance reserves inventory, then authorizes payment, then confirms the... → Readers can observe intermediate states while the saga runs

EXAMPLE FROM THE ORIGINAL GUIDE

Order acceptance reserves inventory, then authorizes payment, then confirms the order. A payment rejection leads to reservation release and order rejection through later local operations.

IMPLEMENTATION SKETCH *illustrative*

```
ACCEPTED -> RESERVED -> PAYMENT_UNKNOWN
                  -> AUTHORIZED -> CONFIRMED
```

INTERVIEWER'S FOLLOW-UP

Which steps are compensatable, retryable or irreversible?

Answer → Reserve/release stock is compensatable; status checks and stable-key authorization attempts are retryable; shipped goods or captured funds require separate irreversible-state policies.

COMMON INTERVIEW TRAP: Readers can observe intermediate states while the saga runs.

MY ONE-LINE RECAP _____

Sagas and workflow recovery

Q052. How do choreography and orchestration differ?

THE 30-SECOND ANSWER

Choreography reacts to events; orchestration uses an explicit coordinator to track the steps. Both need error recovery.

CONCEPT → REAL EXAMPLE → RISK

Choreography lets participants react to events and emit further facts → Inventory reacts to OrderAccepted in a choreographed flow → A coordinator is not a global database transaction manager

EXAMPLE FROM THE ORIGINAL GUIDE

Inventory reacts to OrderAccepted in a choreographed flow. An order workflow coordinator issues ReserveInventory and then AuthorizePayment in an orchestrated flow, tracking each result.

INTERVIEWER'S FOLLOW-UP

When does an event chain become difficult to operate?

Answer → It becomes hard when no single view explains progress, event chains branch heavily, compensation is hidden or operators cannot locate a stuck order.

COMMON INTERVIEW TRAP: A coordinator is not a global database transaction manager.

MY ONE-LINE RECAP _____

Sagas and workflow recovery

Q053. Is compensation the same as rollback?

THE 30-SECOND ANSWER

Rollback undoes an uncommitted database change. Compensation is a later business action and may not restore the original world exactly.

CONCEPT → REAL EXAMPLE → RISK

No	→ Refunding a captured payment does not erase the original charge or guarantee…	→ Compensation success must be observed and recorded, not assumed

EXAMPLE FROM THE ORIGINAL GUIDE

Refunding a captured payment does not erase the original charge or guarantee instant customer funds. Cancelling a shipped order may require a return rather than a simple state reversal.

IMPLEMENTATION SKETCH *illustrative*

```
// Compensation is another committed business action.
refund(originalPaymentId, refundOperationId);
```

INTERVIEWER'S FOLLOW-UP

What do you do if refund succeeds but its response is lost?

Answer → Keep the refund ID and UNKNOWN outcome, query the provider or reconcile callbacks, and never issue a new refund identity until the first outcome is resolved.

COMMON INTERVIEW TRAP: Compensation success must be observed and recorded, not assumed.

MY ONE-LINE RECAP _____

Sagas and workflow recovery

Q054. How should a saga handle an unknown payment result?

THE 30-SECOND ANSWER

A timed-out charge belongs in PAYMENT_UNKNOWN until the payment owner confirms success or rejection. Do not guess from elapsed time.

CONCEPT → REAL EXAMPLE → RISK

Persist an explicit uncertain state and the stable payment operation... → Payment times out after inventory reservation → Cancelling too early can leave a charged but unfulfillable order

EXAMPLE FROM THE ORIGINAL GUIDE

Payment times out after inventory reservation. The order becomes PAYMENT_UNKNOWN. A later confirmed authorization advances it; a definitive rejection releases stock according to policy.

IMPLEMENTATION SKETCH *illustrative*

```
// Do not turn a timeout into rejection.
state = PAYMENT_UNKNOWN;
reconcile(paymentOperationId);
```

INTERVIEWER'S FOLLOW-UP

How do late callbacks interact with reservation expiry?

Answer → Accept callbacks using stable IDs and versions. If stock expired but money is confirmed, follow a documented re-reserve/refund path; never blindly confirm stock.

COMMON INTERVIEW TRAP: Cancelling too early can leave a charged but unfulfillable order.

MY ONE-LINE RECAP _____

Sagas and workflow recovery

Q055. How does a workflow survive coordinator crashes?

THE 30-SECOND ANSWER

Store workflow progress and intended commands durably so another worker can resume after a crash.

CONCEPT → REAL EXAMPLE → RISK

Persist state, correlation identities, expected versions and pending... → The coordinator stores WAITING_PAYMENT plus an outbox command in one transaction → An in-memory state machine is only a teaching model of this behavior

EXAMPLE FROM THE ORIGINAL GUIDE

> The coordinator stores WAITING_PAYMENT plus an outbox command in one transaction. It crashes before delivery; a relay later sends the command, and the restarted workflow accepts the response once.

INTERVIEWER'S FOLLOW-UP

How do two recovery workers avoid advancing the same saga twice?

Answer → Use optimistic state-version updates or atomic claims so only one worker advances the expected state; make duplicate commands and callbacks idempotent.

COMMON INTERVIEW TRAP: An in-memory state machine is only a teaching model of this behavior.

MY ONE-LINE RECAP _____

CHAPTER 12/24

Transactional outbox and idempotent consumers

Make event intent durable and consumer effects repeat-safe.

Five interview questions

Q056. What is the dual-write problem?

Q057. How does a transactional outbox solve that problem?

Q058. Why can an outbox relay publish duplicates?

Q059. How do polling and change-data-capture outbox relays differ?

Q060. How do you implement an idempotent message consumer?

Your challenge

Explain one correct operation, one failure path and the evidence that would tell you the design is working.

Transactional outbox and idempotent consumers

Q056. What is the dual-write problem?

THE 30-SECOND ANSWER

Changing the database and publishing an event are two separate effects. A crash between them can lose events or announce uncommitted facts.

CONCEPT → REAL EXAMPLE → RISK

Writing business data and publishing a message to a separate system can... → Order commits but the broker is unavailable, so no OrderAccepted event reaches... → A try/catch around two independent writes is not atomicity

EXAMPLE FROM THE ORIGINAL GUIDE

Order commits but the broker is unavailable, so no OrderAccepted event reaches inventory. Reversing the order allows consumers to reserve stock for an order that was never committed.

IMPLEMENTATION SKETCH *illustrative*

```
BEGIN;
INSERT INTO orders (...);
INSERT INTO outbox (...);
COMMIT;
```

INTERVIEWER'S FOLLOW-UP

Where could you crash the process to expose the defect?

Answer → Crash after DB commit but before broker send, and after send but before local commit. Test whether each history leaves data and messages consistent.

COMMON INTERVIEW TRAP: A try/catch around two independent writes is not atomicity.

MY ONE-LINE RECAP _____

Transactional outbox and idempotent consumers

Q057. How does a transactional outbox solve that problem?

THE 30-SECOND ANSWER

Write the domain row and an outbox row in the same database transaction. A background relay sends the committed event later.

CONCEPT → REAL EXAMPLE → RISK

Store the business change and an event record in the same local database…	→ The order transaction inserts order o-71 and outbox event e-90	→ The outbox transaction does not include the external broker by magic

EXAMPLE FROM THE ORIGINAL GUIDE

The order transaction inserts order o-71 and outbox event e-90. A rollback removes both; a commit makes both durable. Broker downtime delays delivery without losing the intent.

IMPLEMENTATION SKETCH *illustrative*

```
// One transaction persists fact + event intent:
orders.save(order);
outbox.save(new OrderAccepted(order.id()));
```

INTERVIEWER'S FOLLOW-UP

Which alert detects an increasing age of unpublished records?

Answer → Alert on oldest unpublished outbox age, growing undelivered count, repeated relay failures and deviation from the publication-delay objective.

COMMON INTERVIEW TRAP: The outbox transaction does not include the external broker by magic.

MY ONE-LINE RECAP _____

Transactional outbox and idempotent consumers

Q058. Why can an outbox relay publish duplicates?

THE 30-SECOND ANSWER

A relay can send an event, then crash before marking it sent. It sends it again after restart, so consumers must deduplicate.

CONCEPT → REAL EXAMPLE → RISK

The relay can publish successfully and crash before persisting that success → Event e-90 reaches inventory, but the relay dies before marking it delivered → A delivered flag is not proof of globally exactly-once effects

EXAMPLE FROM THE ORIGINAL GUIDE

Event e-90 reaches inventory, but the relay dies before marking it delivered. Its next run republishes e-90; inventory recognizes the same event identity and does not reserve stock again.

IMPLEMENTATION SKETCH *illustrative*

```
publish(event); // succeeds
crash();        // before marking published
// Recovery can send the same event again.
```

INTERVIEWER'S FOLLOW-UP

How would you test the publish-before-mark crash window?

Answer → Force a crash immediately after broker acknowledgement but before delivery marking. Restart relay and assert exactly one local business effect after duplicate delivery.

COMMON INTERVIEW TRAP: A delivered flag is not proof of globally exactly-once effects.

MY ONE-LINE RECAP _____

Transactional outbox and idempotent consumers

Q059. How do polling and change-data-capture outbox relays differ?

THE 30-SECOND ANSWER

Polling reads outbox rows in batches; CDC reads committed changes from the database log. Both must recover offsets and preserve relevant order.

CONCEPT → REAL EXAMPLE → RISK

Polling queries eligible outbox records and needs indexes, claims and... → A poller claims a bounded batch → Parallel workers can change order unless the relay and consumer contract...

EXAMPLE FROM THE ORIGINAL GUIDE

A poller claims a bounded batch. A CDC connector observes committed outbox inserts. For either, use an aggregate sequence when consumers must detect missing or reordered order transitions.

INTERVIEWER'S FOLLOW-UP

What happens if the connector is down longer than log retention?

Answer → If the database log was truncated past the connector offset, automatic replay may be impossible. Rebuild safely from retained outbox/snapshot records and reconcile missing events.

COMMON INTERVIEW TRAP: Parallel workers can change order unless the relay and consumer contract preserve it.

MY ONE-LINE RECAP _____

Transactional outbox and idempotent consumers

Q060. How do you implement an idempotent message consumer?

THE 30-SECOND ANSWER

Save processed-message identity and business update in the same local transaction. Replays then see the existing key and skip the effect.

CONCEPT → REAL EXAMPLE → RISK

Record the message identity and its local business effect atomically in the… → A billing consumer inserts processed event e-90 and updates its ledger in one… → Deduplication retention must cover the possible replay horizon

EXAMPLE FROM THE ORIGINAL GUIDE

A billing consumer inserts processed event e-90 and updates its ledger in one transaction. If either fails, both roll back, allowing a safe retry. A remote email is represented by a durable job instead of sent inside this assumption.

IMPLEMENTATION SKETCH *illustrative*

```
BEGIN;
INSERT INTO processed_events (...) ON CONFLICT DO NOTHING;
UPDATE ledger ...; -- only if identity newly claimed
COMMIT;
```

INTERVIEWER'S FOLLOW-UP

When is recording an event before an external call unsafe?

Answer → A recorded 'done' key before a separate remote call can falsely hide a call that never occurred. Use durable pending intent plus remote idempotency/reconciliation.

COMMON INTERVIEW TRAP: Deduplication retention must cover the possible replay horizon.

MY ONE-LINE RECAP _____

PART III

EVENTS, SECURITY & DELIVERY

A message is a contract; replay and security are requirements.

Question 061-090

CHAPTER 13/24

Events, schemas and Kafka topics

An event is a versioned promise to consumers.

Five interview questions

Q061. What is the difference between a command and an event?

Q062. How should an event envelope be designed?

Q063. How do you evolve event schemas safely?

Q064. What ordering does Kafka guarantee?

Q065. How should retry topics and dead-letter queues be used?

Your challenge

Explain one correct operation, one failure path and the evidence that would tell you the design is working.

Events, schemas and Kafka topics

Q061. What is the difference between a command and an event?

THE 30-SECOND ANSWER

A command asks an owner to do something and can be refused. An event says something already happened. Name and publish them accordingly.

CONCEPT → REAL EXAMPLE → RISK

A command requests an action from a responsible owner and can be accepted… → ReserveInventory is a command containing an operation identity → A consumer should not reinterpret a historical fact as a mutable instruction

EXAMPLE FROM THE ORIGINAL GUIDE

ReserveInventory is a command containing an operation identity. InventoryReserved is an event emitted after the reservation commits and includes reservation ID, order ID and relevant version.

IMPLEMENTATION SKETCH *illustrative*

```
record ReserveInventory(String orderId) {} // command
record StockReserved(String orderId) {} // event
```

INTERVIEWER'S FOLLOW-UP

How would you correlate a rejected command with its request?

Answer → Return a command/correlation ID, persist rejection state, and publish a linked outcome event or status so callers can connect it to the request.

COMMON INTERVIEW TRAP: A consumer should not reinterpret a historical fact as a mutable instruction.

MY ONE-LINE RECAP _____

Events, schemas and Kafka topics

Q062. How should an event envelope be designed?

THE 30-SECOND ANSWER

An event envelope carries identity, type, version, aggregate key, timestamp, trace/correlation context and payload. Keep sensitive fields controlled.

CONCEPT → REAL EXAMPLE → RISK

Include a stable event ID, event type, schema version, aggregate identity...

→

OrderAccepted carries eventId, tenantId, orderId, aggregateVersion and accepted...

→

A timestamp alone is a poor unique ID or ordering authority

EXAMPLE FROM THE ORIGINAL GUIDE

OrderAccepted carries eventId, tenantId, orderId, aggregateVersion and accepted totals. Trace metadata helps diagnosis; the event ID supports deduplication and the aggregate version supports ordering checks.

IMPLEMENTATION SKETCH *illustrative*

```
record Envelope(String eventId, String aggregateId,
                int version, String type) {}
```

INTERVIEWER'S FOLLOW-UP

Which fields belong in the partition key rather than only the payload?

Answer → Put a stable aggregate or entity ID in the Kafka record key so related records reach the intended partition and maintain that partition's order.

COMMON INTERVIEW TRAP: A timestamp alone is a poor unique ID or ordering authority.

MY ONE-LINE RECAP _____

Events, schemas and Kafka topics

Q063. How do you evolve event schemas safely?

THE 30-SECOND ANSWER

Event consumers may replay old messages long after a producer changes. Add compatible fields and explicitly plan schema versions.

CONCEPT → REAL EXAMPLE → RISK

Test compatible producer and consumer combinations, preserve field meaning... → Add an optional warehouseZone field with a documented default → An enum addition can fail strict readers

EXAMPLE FROM THE ORIGINAL GUIDE

Add an optional warehouseZone field with a documented default. Changing amount from major currency units to minor units under the same field name is a semantic break even when its numeric type stays valid.

INTERVIEWER'S FOLLOW-UP

Which compatibility direction is required for replaying old events?

Answer → New readers must handle historical old messages during replay: backward compatibility. Test both directions when old readers also consume new publications.

COMMON INTERVIEW TRAP: An enum addition can fail strict readers.

MY ONE-LINE RECAP _____

Events, schemas and Kafka topics

Q064. What ordering does Kafka guarantee?

THE 30-SECOND ANSWER

Kafka orders records within one partition, not across all partitions. Events for one aggregate should use a stable partition key.

CONCEPT → REAL EXAMPLE → RISK

Kafka orders records within a partition, not across an entire... → All order o-71 transitions use the same key and partition → Increasing partition count can change key placement for future records

EXAMPLE FROM THE ORIGINAL GUIDE

All order o-71 transitions use the same key and partition. If payment results are processed concurrently and committed out of order, partition ordering alone has not preserved the order workflow.

IMPLEMENTATION SKETCH *illustrative*

```
producer.send("orders", orderId, event);
// Aggregate ID becomes the Kafka record key.
```

INTERVIEWER'S FOLLOW-UP

How would you migrate partitions without violating aggregate ordering?

Answer → Repartition using a coordinated boundary, drain older ordered records and track sequence versions; two live partitioning schemes can otherwise reorder one aggregate.

COMMON INTERVIEW TRAP: Increasing partition count can change key placement for future records.

MY ONE-LINE RECAP _____

Events, schemas and Kafka topics

Q065. How should retry topics and dead-letter queues be used?

THE 30-SECOND ANSWER

Retry topics delay failures; DLQs isolate messages needing inspection. Both must respect the business ordering requirement.

CONCEPT → REAL EXAMPLE → RISK

Classify failures, bound attempts, and preserve enough original identity... → A malformed notification event reaches a DLQ with its ID and error classification → A DLQ with no review or redrive policy is delayed data loss

EXAMPLE FROM THE ORIGINAL GUIDE

A malformed notification event reaches a DLQ with its ID and error classification. The owner repairs the producer or consumer and redrives it using the original identity to avoid another send.

INTERVIEWER'S FOLLOW-UP

Can later events for the same aggregate safely proceed?

Answer → Only when later events do not depend on the failed event, or a per-aggregate sequence gate prevents applying them early. Otherwise pause or repair that key.

COMMON INTERVIEW TRAP: A DLQ with no review or redrive policy is delayed data loss.

MY ONE-LINE RECAP _____

CHAPTER 14/24

Delivery semantics, consumer groups and replay

Kafka ordering is partition-scoped; external effects are different.

Five interview questions

Q066. How do producer acknowledgement, consumer offset and business commit differ?

Q067. What does exactly-once mean with Kafka?

Q068. How do consumer groups and partitions affect parallelism?

Q069. What is consumer lag, and how should it be interpreted?

Q070. How can you replay events safely?

Your challenge

Explain one correct operation, one failure path and the evidence that would tell you the design is working.

Delivery semantics, consumer groups and replay

Q066. How do producer acknowledgement, consumer offset and business commit differ?

THE 30-SECOND ANSWER

Producer acks concern broker writes, committed offsets concern reading progress, and the business database has its own commit boundary.

CONCEPT → REAL EXAMPLE → RISK

A producer acknowledgement reports the configured broker acceptance outcome → The ledger commits an update, then crashes before committing the Kafka offset → Receiving a record is not proof that its business effect is durable

EXAMPLE FROM THE ORIGINAL GUIDE

The ledger commits an update, then crashes before committing the Kafka offset. Kafka redelivers and the ledger deduplicates the event. Committing the offset first could instead skip an effect after a crash.

IMPLEMENTATION SKETCH *illustrative*

```
// Commit DB effect first; then record offset progress.
// Duplicate consumption must be harmless.
```

INTERVIEWER'S FOLLOW-UP

Which crash windows must your consumer test?

Answer → Crash before effect commit, after effect commit before offset, and after offset before effect. Assert no lost effect or repeated business update.

COMMON INTERVIEW TRAP: Receiving a record is not proof that its business effect is durable.

MY ONE-LINE RECAP _____

Delivery semantics, consumer groups and replay

Q067. What does exactly-once mean with Kafka?

THE 30-SECOND ANSWER

Kafka exactly-once features protect supported Kafka processing/transactions, not any arbitrary external database write.

CONCEPT → REAL EXAMPLE → RISK

Kafka transactions and appropriately configured consumers can atomically... → A Kafka processor reads a record, writes a derived Kafka record and commits... → Exactly-once is incomplete unless the included resources and failure model...

EXAMPLE FROM THE ORIGINAL GUIDE

A Kafka processor reads a record, writes a derived Kafka record and commits offsets in one transaction. A separate credit-card charge still needs a payment operation identity and reconciliation.

IMPLEMENTATION SKETCH *illustrative*

```
UNIQUE (consumer_name, event_id)
// Same transaction as the ledger update.
```

INTERVIEWER'S FOLLOW-UP

How would you achieve one durable ledger effect per event?

Answer → Use a unique consumer/event ID and save it together with the ledger update in one database transaction. Then commit the Kafka offset after success.

COMMON INTERVIEW TRAP: Exactly-once is incomplete unless the included resources and failure model are named.

MY ONE-LINE RECAP _____

Delivery semantics, consumer groups and replay

Q068. How do consumer groups and partitions affect parallelism?

THE 30-SECOND ANSWER

A consumer group divides partitions among active members. More consumers than partitions do not increase parallel reading of that topic.

CONCEPT → REAL EXAMPLE → RISK

Within a traditional consumer-group assignment, each partition is assigned... → A topic with six partitions supports up to six actively assigned consumers in one... → Newer Kafka consumption models may differ; identify the mode in use

EXAMPLE FROM THE ORIGINAL GUIDE

A topic with six partitions supports up to six actively assigned consumers in one such group. Two different groups each receive the stream for separate business purposes.

INTERVIEWER'S FOLLOW-UP

Why might adding consumers fail to reduce lag?

Answer → You may already have one consumer per partition, or be blocked by a hot key, slow DB, rebalances or expensive messages. Measure per-partition lag and processing time.

COMMON INTERVIEW TRAP: Newer Kafka consumption models may differ; identify the mode in use.

MY ONE-LINE RECAP _____

Delivery semantics, consumer groups and replay

Q069. What is consumer lag, and how should it be interpreted?

THE 30-SECOND ANSWER

Consumer lag counts how far reading is behind writing, but does not explain why the consumer is slow or whether events are useful.

CONCEPT → REAL EXAMPLE → RISK

Lag measures distance between selected log progress and consumer progress	→	A consumer commits early and hands records to an executor → Lag is a symptom, not a root-cause diagnosis

EXAMPLE FROM THE ORIGINAL GUIDE

A consumer commits early and hands records to an executor. Offset lag looks low while durable notification completion is hours behind. Track oldest outstanding job age and effect completion separately.

IMPLEMENTATION SKETCH *illustrative*

```
// Lag alone is not enough:
monitor(partitionLag, oldestEventAge, errorRate);
```

INTERVIEWER'S FOLLOW-UP

Which measurements distinguish insufficient capacity from poison messages?

Answer → Compare lag by partition, oldest-record age, failures/retries, poison-message IDs, processing throughput, broker fetch latency and downstream saturation.

COMMON INTERVIEW TRAP: Lag is a symptom, not a root-cause diagnosis.

MY ONE-LINE RECAP _____

Delivery semantics, consumer groups and replay

Q070. How can you replay events safely?

THE 30-SECOND ANSWER

Replay is a deliberate reprocessing of retained facts. Consumers must tolerate duplicates and control their effect destination.

CONCEPT → REAL EXAMPLE → RISK

Choose the replay purpose and horizon, preserve identities and isolate side...

→

Rebuild an order-search projection in a new index from retained events

→

Replay from a retained topic may still be incomplete after compaction or...

EXAMPLE FROM THE ORIGINAL GUIDE

Rebuild an order-search projection in a new index from retained events. Do not resend every historical customer email merely because a new group reads the stream.

INTERVIEWER'S FOLLOW-UP

How would you verify a rebuilt projection before cutover?

Answer → Rebuild into an isolated projection, compare counts, totals, versions and sampled records with owner data; switch reads only after checks and a catch-up barrier.

COMMON INTERVIEW TRAP: Replay from a retained topic may still be incomplete after compaction or expiry.

MY ONE-LINE RECAP _____

CHAPTER 15/24

CQRS, projections and caching

Read models have a freshness and rebuild contract.

Five interview questions

Q071. How do CQRS and event sourcing differ?

Q072. How does a projection handle duplicates and missing events?

Q073. How do you query across service-owned data?

Q074. How does cache-aside work, and where do races appear?

Q075. How do you prevent a cache stampede?

Your challenge

Explain one correct operation, one failure path and the evidence that would tell you the design is working.

CQRS, projections and caching

Q071. How do CQRS and event sourcing differ?

THE 30-SECOND ANSWER

CQRS separates write and read models. Event sourcing stores events as the primary history used to rebuild state. They are not the same pattern.

CONCEPT → REAL EXAMPLE → RISK

CQRS separates command and query models; it does not require event sourcing...

→

A normal order table feeds a denormalized search projection: CQRS without event...

→

An audit table beside current state is not automatically event sourcing

EXAMPLE FROM THE ORIGINAL GUIDE

A normal order table feeds a denormalized search projection: CQRS without event sourcing. An event-sourced order stores accepted and cancelled events as its authoritative history.

INTERVIEWER'S FOLLOW-UP

When would historical state reconstruction justify the extra complexity?

Answer → Choose event sourcing when audit, temporal reconstruction or alternate models justify storage, schema evolution and replay operations—not only because events are fashionable.

COMMON INTERVIEW TRAP: An audit table beside current state is not automatically event sourcing.

MY ONE-LINE RECAP _____

CQRS, projections and caching

Q072. How does a projection handle duplicates and missing events?

THE 30-SECOND ANSWER

A projection records the last applied sequence, skips duplicates and detects missing versions before applying dependent deltas.

CONCEPT → REAL EXAMPLE → RISK

Give events stable identities and, where required, per-aggregate versions → A balance projection expects version 8 but receives 10 → Higher version does not automatically make a partial update safe

EXAMPLE FROM THE ORIGINAL GUIDE

A balance projection expects version 8 but receives 10. Applying a delta would miss version 9, so it requests replay or repair. A full-state snapshot may allow a different explicit policy.

IMPLEMENTATION SKETCH *illustrative*

```
if (version <= currentVersion) return; // duplicate
if (version != currentVersion + 1) repairGap();
```

INTERVIEWER'S FOLLOW-UP

Where do you store projection version and processed identity?

Answer → Store the projection's aggregate version and any processed event identity in the same transaction as the projected values; repair gaps through replay.

COMMON INTERVIEW TRAP: Higher version does not automatically make a partial update safe.

MY ONE-LINE RECAP _____

CQRS, projections and caching

Q073. How do you query across service-owned data?

THE 30-SECOND ANSWER

Cross-service reports usually use dedicated read models or coordinated queries. Define freshness and field ownership explicitly.

CONCEPT → REAL EXAMPLE → RISK

Use API composition for bounded live joins or a maintained read model for... → An order page composes one order with payment status → Large fan-out joins can make one slow service dominate latency

EXAMPLE FROM THE ORIGINAL GUIDE

An order page composes one order with payment status. A monthly revenue report uses a governed reporting store instead of issuing a remote join for every customer across live services.

INTERVIEWER'S FOLLOW-UP

What makes a report trustworthy when source events are late?

Answer → Include source watermark/version, lag/last updated timestamps, reconciliation counts and correction procedures. Mark late or missing input instead of pretending completeness.

COMMON INTERVIEW TRAP: Large fan-out joins can make one slow service dominate latency.

MY ONE-LINE RECAP _____

CQRS, projections and caching

Q074. How does cache-aside work, and where do races appear?

THE 30-SECOND ANSWER

Cache-aside loads from the owner on a miss and caches the result. Concurrent updates may race with an old load that writes stale data back.

CONCEPT → REAL EXAMPLE → RISK

Read the cache, load from the authoritative store on a miss and populate it…	→	Tenant A's product price is cached under tenant A and product ID → A TTL limits exposure duration; it is not strong consistency

EXAMPLE FROM THE ORIGINAL GUIDE

Tenant A's product price is cached under tenant A and product ID. After a price update, invalidation may race with an earlier read; a version check prevents an older value from replacing a newer entry.

IMPLEMENTATION SKETCH *illustrative*

```
// Always scope cache keys to the verified tenant.
String key = tenantId + ":" + productId;
```

INTERVIEWER'S FOLLOW-UP

How would you test the stale-repopulation race?

Answer → Pause a cache miss after its DB read, update DB and invalidate, then allow stale load to populate. Assert version checks or invalidation prevent stale replacement.

COMMON INTERVIEW TRAP: A TTL limits exposure duration; it is not strong consistency.

MY ONE-LINE RECAP _____

CQRS, projections and caching

Q075. How do you prevent a cache stampede?

THE 30-SECOND ANSWER

A cache stampede is many identical misses rebuilding one expensive value. Coalesce loads, bound concurrency and spread expiration times.

CONCEPT → REAL EXAMPLE → RISK

Reduce simultaneous misses with per-key single-flight loading, controlled… → Ten replicas see a popular entry expire → A lock with a long wait can turn a stampede into a queueing outage

EXAMPLE FROM THE ORIGINAL GUIDE

Ten replicas see a popular entry expire. Jitter spreads expiries, while a bounded loader protects the database. Optional catalog data can serve a marked stale entry during refresh; authorization cannot use the same policy casually.

IMPLEMENTATION SKETCH *illustrative*

```
// Only one refresh per key; bounded stale fallback.
computeIfAbsent(key, ignored -> loadFromOwner());
```

INTERVIEWER'S FOLLOW-UP

What happens when refresh fails repeatedly?

Answer → After repeated refresh failure, use a bounded stale-serve policy where safe, exponential retry, breaker and per-key admission; never overload the origin.

COMMON INTERVIEW TRAP: A lock with a long wait can turn a stampede into a queueing outage.

MY ONE-LINE RECAP _____

CHAPTER 16/24

Security across service boundaries

Authentication does not replace tenant authorization.

Five interview questions

Q076. How do OAuth, OIDC and JWT relate to microservices?

Q077. Why must services enforce authorization even behind a gateway?

Q078. What do mTLS and workload identity provide?

Q079. When should client credentials or user-token propagation be used?

Q080. How do you reduce secret leakage and server-side request forgery?

Your challenge

Explain one correct operation, one failure path and the evidence that would tell you the design is working.

Security across service boundaries

Q076. How do OAuth, OIDC and JWT relate to microservices?

THE 30-SECOND ANSWER

OAuth is an authorization framework; OIDC adds login identity; JWT is one token format. A valid signature alone does not authorize a business action.

CONCEPT → REAL EXAMPLE → RISK

OAuth is an authorization framework for delegated or application access → A user signs in through OIDC, then presents an access token to an order resource… → An ID token is not interchangeable with an API access token

EXAMPLE FROM THE ORIGINAL GUIDE

A user signs in through OIDC, then presents an access token to an order resource server. The service checks signature or token validity, issuer, audience, time constraints and the relevant permissions.

IMPLEMENTATION SKETCH *illustrative*

```
// API token validation: issuer, audience, time, signature.
// Then check tenant permission in the domain service.
```

INTERVIEWER'S FOLLOW-UP

Which validation rejects a token issued for another API?

Answer → Check intended API audience as well as issuer, signature, expiry and required scopes. Reject a valid token whose aud names another resource server.

COMMON INTERVIEW TRAP: An ID token is not interchangeable with an API access token.

MY ONE-LINE RECAP _____

Security across service boundaries

Q077. Why must services enforce authorization even behind a gateway?

THE 30-SECOND ANSWER

A gateway may authenticate a caller, but only the domain service knows whether that caller owns a particular order or tenant resource.

CONCEPT → REAL EXAMPLE → RISK

The gateway's policy may authenticate requests or protect routes, but... → An authenticated user can reach GET /orders/id, but the order owner must verify... → A user-controlled tenant header must not override verified tenant authority

EXAMPLE FROM THE ORIGINAL GUIDE

An authenticated user can reach GET /orders/id, but the order owner must verify that this user or service may read that tenant's particular order. Route-level login is insufficient.

IMPLEMENTATION SKETCH *illustrative*

```
if (!resource.tenantId().equals(currentTenant))
    throw new AccessDeniedException("tenant");
```

INTERVIEWER'S FOLLOW-UP

How would you test cross-tenant resource access?

Answer → Call with tenant A's token against tenant B's ID and assert denial across get/update/export endpoints, including cached and batch paths.

COMMON INTERVIEW TRAP: A user-controlled tenant header must not override verified tenant authority.

MY ONE-LINE RECAP _____

Security across service boundaries

Q078. What do mTLS and workload identity provide?

THE 30-SECOND ANSWER

mTLS authenticates connected workloads and encrypts transport. It does not prove the end user's permission to read a specific invoice.

CONCEPT → REAL EXAMPLE → RISK

mTLS authenticates peers at the transport boundary and encrypts traffic;... → Only an approved order workload can establish the allowed payment connection → Encryption does not prove that the requested business action is authorized

EXAMPLE FROM THE ORIGINAL GUIDE

Only an approved order workload can establish the allowed payment connection. The payment service still checks whether the request is permitted for the tenant and operation being charged.

INTERVIEWER'S FOLLOW-UP

How do you rotate credentials without interrupting all traffic?

Answer → Overlap old and new certificates/credentials, rotate connections gradually, verify new handshakes, then retire old trust material without restarting everything at once.

COMMON INTERVIEW TRAP: Encryption does not prove that the requested business action is authorized.

MY ONE-LINE RECAP _____

Security across service boundaries

Q079. When should client credentials or user-token propagation be used?

THE 30-SECOND ANSWER

Client credentials identify service-to-service work; delegated user context is used only when user identity truly matters. Never blindly forward every token.

CONCEPT → REAL EXAMPLE → RISK

Client credentials represent an application's own authority → A nightly settlement job uses service authority → Do not invent trusted claims in headers unless the receiving boundary...

EXAMPLE FROM THE ORIGINAL GUIDE

A nightly settlement job uses service authority. A user cancelling an order carries user context under a supported delegation contract, while the downstream service independently verifies its permission.

IMPLEMENTATION SKETCH *illustrative*

```
// Audit both identities:
log(workloadId, userSubject, tenantId, operationId);
```

INTERVIEWER'S FOLLOW-UP

How do you audit both the calling workload and the end user?

Answer → Record workload identity and authorized end-user subject separately with trace ID, tenant, action, decision and outcome; minimize sensitive token logging.

COMMON INTERVIEW TRAP: Do not invent trusted claims in headers unless the receiving boundary verifies them.

MY ONE-LINE RECAP _____

Security across service boundaries

Q080. How do you reduce secret leakage and server-side request forgery?

THE 30-SECOND ANSWER

Keep secrets out of code/logs and stop services from fetching arbitrary URLs on behalf of clients. Validate destinations and network egress.

CONCEPT → REAL EXAMPLE → RISK

Use approved secret storage, restrict diagnostic exposure and redact...

→

A document service accepts object IDs mapped to approved storage, not arbitrary...

→

String-prefix URL checks can be bypassed by parsing or redirect behavior

EXAMPLE FROM THE ORIGINAL GUIDE

A document service accepts object IDs mapped to approved storage, not arbitrary URLs that can target internal admin services. Access tokens and signed URLs are excluded from ordinary request logs.

INTERVIEWER'S FOLLOW-UP

Which egress controls would support the application checks?

Answer → Use egress allowlists, private DNS/VPC rules, proxy filtering, metadata endpoint protection and restrictive workload network policies alongside URL validation.

COMMON INTERVIEW TRAP: String-prefix URL checks can be bypassed by parsing or redirect behavior.

MY ONE-LINE RECAP _____

CHAPTER 17/24

Observability and reliability objectives

Observe progress of the real business operation.

Five interview questions

Q081. What do logs, metrics and traces each tell you?

Q082. How is trace context propagated across services?

Q083. What are SLI, SLO, SLA and an error budget?

Q084. How does error-budget burn-rate alerting work?

Q085. Why is metric cardinality dangerous?

Your challenge

Explain one correct operation, one failure path and the evidence that would tell you the design is working.

Observability and reliability objectives

Q081. What do logs, metrics and traces each tell you?

THE 30-SECOND ANSWER

Logs explain events, metrics show trends and alerts, and traces join the path of one request. Use all three for production diagnosis.

CONCEPT → REAL EXAMPLE → RISK

Logs describe selected events, metrics aggregate measurements over... → Checkout error rate rises; traces show time spent waiting for payment connections,... → A dashboard without a user-facing success definition can look healthy during...

EXAMPLE FROM THE ORIGINAL GUIDE

Checkout error rate rises; traces show time spent waiting for payment connections, and pool metrics confirm saturation. Logs identify the release and error classification without exposing card data.

IMPLEMENTATION SKETCH *illustrative*

```
log.info("orderId={} state={}", id, state);
// Add traces + age-of-pending-order metrics.
```

INTERVIEWER'S FOLLOW-UP

Which signal would reveal a successfully accepted but stuck order?

Answer → Alert on orders remaining PENDING too long, workflow age, missing transition events and outbox/consumer backlog age—not only API error rate.

COMMON INTERVIEW TRAP: A dashboard without a user-facing success definition can look healthy during an outage.

MY ONE-LINE RECAP _____

Observability and reliability objectives

Q082. How is trace context propagated across services?

THE 30-SECOND ANSWER

Trace context follows requests through services and messages so spans can be correlated. A business operation ID must outlive trace retention.

CONCEPT → REAL EXAMPLE → RISK

Instrumentation injects and extracts a supported context, commonly W3C...	→ Order calls inventory with trace context and publishes an event with causation...	→ A user-controlled baggage field is not trusted authorization or unlimited...

EXAMPLE FROM THE ORIGINAL GUIDE

Order calls inventory with trace context and publishes an event with causation metadata. A later worker starts a causally linked processing span while retaining order ID for business recovery.

INTERVIEWER'S FOLLOW-UP

When is a span link more appropriate than a single parent?

Answer → Use a span link when batch or asynchronous work has multiple triggering contexts, or when the processing relationship is not a simple single-parent child.

COMMON INTERVIEW TRAP: A user-controlled baggage field is not trusted authorization or unlimited metric metadata.

MY ONE-LINE RECAP _____

Observability and reliability objectives

Q083. What are SLI, SLO, SLA and an error budget?

THE 30-SECOND ANSWER

An SLI is what you measure, an SLO is your target, an SLA is a customer agreement, and an error budget permits limited misses.

CONCEPT → REAL EXAMPLE → RISK

An SLI measures a chosen service outcome; an SLO sets its target over a... → For eligible checkout attempts, 99 → A process uptime metric is not necessarily a checkout-success SLI

EXAMPLE FROM THE ORIGINAL GUIDE

For eligible checkout attempts, 99.9% must complete successfully under one second over a rolling 30-day window. Invalid customer input may be excluded by a documented policy, not hidden after an incident.

IMPLEMENTATION SKETCH *illustrative*

```
SLI = goodBusinessOperations / eligibleOperations
errorBudget = (1 - SLO) * eligibleOperations
```

INTERVIEWER'S FOLLOW-UP

What should count as good when work completes asynchronously?

Answer → Count durable acceptance plus completion within the published window as good when the product promises final completion; report pending/out-of-window separately.

COMMON INTERVIEW TRAP: A process uptime metric is not necessarily a checkout-success SLI.

MY ONE-LINE RECAP _____

Observability and reliability objectives

Q084. How does error-budget burn-rate alerting work?

THE 30-SECOND ANSWER

Burn-rate compares how fast current bad events use up the allowed error budget. Fast burn needs quick action; slower burn needs investigation.

CONCEPT → REAL EXAMPLE → RISK

Burn rate compares the observed bad-event fraction with the fraction… → With a 99 → Percentile latency and request success can require separate SLIs

EXAMPLE FROM THE ORIGINAL GUIDE

With a 99.9% objective, the allowed bad fraction is 0.1%. A sustained 1% bad fraction has a burn rate of 10, consuming budget ten times faster than the target rate.

INTERVIEWER'S FOLLOW-UP

What action should an alert trigger, and who owns it?

Answer → Page the on-call owner for rapid user-impacting burn with a runbook; slow burn can create an actionable ticket and pause risky releases.

COMMON INTERVIEW TRAP: Percentile latency and request success can require separate SLIs.

MY ONE-LINE RECAP _____

Observability and reliability objectives

Q085. Why is metric cardinality dangerous?

THE 30-SECOND ANSWER

High-cardinality labels create huge metric series and cost. Request ID should usually be in logs/traces, not metric labels.

CONCEPT → REAL EXAMPLE → RISK

Each distinct label combination can create another time series → Record latency by normalized route, service and bounded outcome class → Even individually bounded labels can produce a large cross product

EXAMPLE FROM THE ORIGINAL GUIDE

Record latency by normalized route, service and bounded outcome class. Store order o-71 in a trace or structured log rather than making every order a Prometheus label value.

INTERVIEWER'S FOLLOW-UP

How would you review telemetry cost before adding a label?

Answer → Estimate series count as the product of label values, sample expected tenant/endpoint volumes, define retention and compare storage/query spend before rollout.

COMMON INTERVIEW TRAP: Even individually bounded labels can produce a large cross product.

MY ONE-LINE RECAP _____

CHAPTER 18/24

Testing distributed contracts and failures

A test only proves the boundary it actually exercises.

Five interview questions

Q086. What testing layers are useful for microservices?

Q087. What is consumer-driven contract testing?

Q088. What can Testcontainers prove?

Q089. Which failure tests are essential for event processing?

Q090. How should chaos and load tests be designed?

Your challenge

Explain one correct operation, one failure path and the evidence that would tell you the design is working.

Testing distributed contracts and failures

Q086. What testing layers are useful for microservices?

THE 30-SECOND ANSWER

Use unit, integration, contract and end-to-end tests, plus targeted crash tests. Each proves a different boundary.

CONCEPT → REAL EXAMPLE → RISK

Use unit tests for domain decisions, adapter tests for protocol mapping,... → Test reservation arithmetic as a unit, affected-row behavior against the target... → Test doubles can accidentally reproduce the same mistaken assumption as the...

EXAMPLE FROM THE ORIGINAL GUIDE

Test reservation arithmetic as a unit, affected-row behavior against the target database, API fields as a contract and one checkout journey with controlled payment behavior.

IMPLEMENTATION SKETCH *illustrative*

```
@Test void rollbackRemovesOutbox() {
  // Assert both rows absent after forced rollback.
}
```

INTERVIEWER'S FOLLOW-UP

Which test proves that a rollback removes the outbox record?

Answer → An actual database integration test starts the transaction, forces rollback and verifies both order and outbox rows are absent afterward.

COMMON INTERVIEW TRAP: Test doubles can accidentally reproduce the same mistaken assumption as the code.

MY ONE-LINE RECAP _____

Testing distributed contracts and failures

Q087. What is consumer-driven contract testing?

THE 30-SECOND ANSWER

Consumer-driven contracts capture what a consumer requires from a provider. Provider verification checks those promises against a real provider build.

CONCEPT → REAL EXAMPLE → RISK

Consumers specify the aspects of a provider contract they actually rely on;... → The checkout consumer expects inventory reservation success to contain... → A contract verified only against a mock provider proves little about the...

EXAMPLE FROM THE ORIGINAL GUIDE

The checkout consumer expects inventory reservation success to contain reservationId and expiresAt. Provider verification fails if a new release removes expiresAt without an agreed migration.

IMPLEMENTATION SKETCH *illustrative*

```
// Provider CI must verify active consumer contracts
// BEFORE rollout.
```

INTERVIEWER'S FOLLOW-UP

Where should provider verification block a release?

Answer → Block provider release in CI when the proposed artifact breaks active consumer contracts; include deployed consumer versions and compatibility windows.

COMMON INTERVIEW TRAP: A contract verified only against a mock provider proves little about the real implementation.

MY ONE-LINE RECAP _____

Testing distributed contracts and failures

Q088. What can Testcontainers prove?

THE 30-SECOND ANSWER

Testcontainers gives disposable real infrastructure components in integration tests. It cannot reproduce every cloud configuration, network rule or production failure.

CONCEPT → REAL EXAMPLE → RISK

Testcontainers can run real infrastructure dependencies for integration...

→

A PostgreSQL test runs two competing stock reservations and checks that no...

→

An in-memory database may have different locking and SQL semantics

EXAMPLE FROM THE ORIGINAL GUIDE

A PostgreSQL test runs two competing stock reservations and checks that no negative inventory is committed. A Kafka test restarts processing around a commit window and checks one durable ledger effect.

INTERVIEWER'S FOLLOW-UP

Which production configuration differences remain after the test?

Answer → Account for managed database settings, TLS/IAM, quotas, multi-AZ behavior, broker retention, Kubernetes networking and scale in separate environment checks.

COMMON INTERVIEW TRAP: An in-memory database may have different locking and SQL semantics.

MY ONE-LINE RECAP _____

Testing distributed contracts and failures

Q089. Which failure tests are essential for event processing?

THE 30-SECOND ANSWER

Crash testing exposes lost updates, duplicate effects and offset races that normal integration tests miss.

CONCEPT → REAL EXAMPLE → RISK

Test duplicate deliveries, crashes before and after business commit,...	→	A payment double records a charge but drops its response → Happy-path exactly-once assertions do not cover ambiguous outcomes

EXAMPLE FROM THE ORIGINAL GUIDE

A payment double records a charge but drops its response. A repeated request with the same key returns that recorded charge; a test fails if the order creates a second charge identity.

IMPLEMENTATION SKETCH *illustrative*

```
// Crash after DB commit, before message/offset ACK.
// Replay should create one ledger effect.
```

INTERVIEWER'S FOLLOW-UP

Where would you inject a crash to expose early offset commits?

Answer → Commit the Kafka offset before writing the business database result, kill the consumer and restart. The test should show why that order can lose an effect.

COMMON INTERVIEW TRAP: Happy-path exactly-once assertions do not cover ambiguous outcomes.

MY ONE-LINE RECAP _____

Testing distributed contracts and failures

Q090. How should chaos and load tests be designed?

THE 30-SECOND ANSWER

Chaos tests test named failure hypotheses; load tests measure capacity under known conditions. Keep abort criteria and measurable customer impact.

CONCEPT → REAL EXAMPLE → RISK

Choose a bounded experiment, a hypothesis and a measurable steady-state... → Delay the recommendation dependency while checking that purchase success remains... → A random outage without a hypothesis often produces little learning

EXAMPLE FROM THE ORIGINAL GUIDE

Delay the recommendation dependency while checking that purchase success remains within its objective and recommendation concurrency stays bounded. Restore it and verify queue and circuit recovery.

INTERVIEWER'S FOLLOW-UP

What would falsify your claimed isolation between workloads?

Answer → Show recommendations saturating checkout connections or raising checkout error rate despite the claimed bulkhead; that falsifies the isolation claim.

COMMON INTERVIEW TRAP: A random outage without a hypothesis often produces little learning.

MY ONE-LINE RECAP _____

PART IV

PRODUCTION & ARCHITECT JUDGMENT

Prove an architecture with release and recovery evidence.

Question 091-120

CHAPTER 19/24

Deployment, Kubernetes and schema changes

Safe release is a compatibility problem as well as an image change.

Five interview questions

Q091. What should a microservice delivery pipeline verify?

Q092. How do startup, readiness and liveness probes differ in Kubernetes?

Q093. How do rolling, blue-green and canary deployment differ?

Q094. What is an expand-contract database migration?

Q095. What does graceful shutdown require for a service?

Your challenge

Explain one correct operation, one failure path and the evidence that would tell you the design is working.

Deployment, Kubernetes and schema changes

Q091. What should a microservice delivery pipeline verify?

THE 30-SECOND ANSWER

A release pipeline should check build quality, compatibility, dependencies, security, deployment health and real business outcomes.

CONCEPT → REAL EXAMPLE → RISK

Build a traceable immutable candidate, verify domain and boundary tests,... → The same image digest moves from staging to production after verification → A green build does not prove production settings or dependencies are correct

EXAMPLE FROM THE ORIGINAL GUIDE

The same image digest moves from staging to production after verification. Production selects its own endpoints and secrets, and a checkout synthetic check verifies the deployed behavior.

IMPLEMENTATION SKETCH *illustrative*

```
mvn test && contractTest && integrationTest
// Then canary business-outcome checks.
```

INTERVIEWER'S FOLLOW-UP

Which check would block an incompatible event release?

Answer → An event-schema consumer contract test must fail if active or replaying consumers cannot deserialize or correctly interpret the new event.

COMMON INTERVIEW TRAP: A green build does not prove production settings or dependencies are correct.

MY ONE-LINE RECAP _____

Deployment, Kubernetes and schema changes

Q092. How do startup, readiness and liveness probes differ in Kubernetes?

THE 30-SECOND ANSWER

Startup gates initial initialization; readiness controls traffic; liveness requests restart when local process recovery is likely to help.

CONCEPT → REAL EXAMPLE → RISK

Startup probes allow initialization before other configured probes take over → A slow schema warm-up needs startup handling → Use the selected Kubernetes version and inspect actual probe and routing...

EXAMPLE FROM THE ORIGINAL GUIDE

A slow schema warm-up needs startup handling. A wedged process may fail liveness. A local inability to serve the API may fail readiness while the team repairs it without unnecessary restart churn.

IMPLEMENTATION SKETCH *illustrative*

```
startupProbe -> initialization complete
readinessProbe -> admit traffic
livenessProbe -> local restart is helpful
```

INTERVIEWER'S FOLLOW-UP

Why is an external payment outage usually a poor liveness check?

Answer → A remote payment outage is outside the pod's repair control. Restarting every pod increases load and hides an otherwise recoverable dependency incident.

COMMON INTERVIEW TRAP: Use the selected Kubernetes version and inspect actual probe and routing behavior.

MY ONE-LINE RECAP _____

Deployment, Kubernetes and schema changes

Q093. How do rolling, blue-green and canary deployment differ?

THE 30-SECOND ANSWER

Rolling replaces replicas gradually, blue-green switches between complete environments, and canary exposes a small audience to a new version.

CONCEPT → REAL EXAMPLE → RISK

Rolling replaces instances gradually, so old and new versions coexist	→ A canary receives a small measured checkout share	→ Reversing traffic does not undo writes made by the candidate

EXAMPLE FROM THE ORIGINAL GUIDE

A canary receives a small measured checkout share. Increased failure or latency triggers reversal, while event consumers and schema changes remain compatible with both releases.

INTERVIEWER'S FOLLOW-UP

What could make a canary comparison misleading?

Answer → Unequal tenants, traffic mix, warmup, cache state or downstream dependencies can skew error/latency comparisons. Match cohorts and compare business outcomes.

COMMON INTERVIEW TRAP: Reversing traffic does not undo writes made by the candidate.

MY ONE-LINE RECAP _____

Deployment, Kubernetes and schema changes

Q094. What is an expand-contract database migration?

THE 30-SECOND ANSWER

Expand-contract adds compatible schema first, runs old and new code together, backfills, then removes obsolete fields after verification.

CONCEPT → REAL EXAMPLE → RISK

First add compatible structure, then deploy code that can coexist with old... → Add delivery_region beside region, backfill safely, update readers and writers in... → A destructive migration can make an otherwise healthy old image unusable

EXAMPLE FROM THE ORIGINAL GUIDE

Add delivery_region beside region, backfill safely, update readers and writers in stages, verify completeness, then remove region in a later release after compatibility checks.

IMPLEMENTATION SKETCH *illustrative*

```
ADD COLUMN new_status TEXT NULL;
-- Backfill in batches, switch readers, contract later.
```

INTERVIEWER'S FOLLOW-UP

How do you prevent a backfill from overwriting a newer live update?

Answer → Read the current row version during backfill and update conditionally; never let an old snapshot overwrite a newer application write.

COMMON INTERVIEW TRAP: A destructive migration can make an otherwise healthy old image unusable.

MY ONE-LINE RECAP _____

Deployment, Kubernetes and schema changes

Q095. What does graceful shutdown require for a service?

THE 30-SECOND ANSWER

Graceful shutdown stops admitting new work, drains or transfers accepted work and releases resources under a deadline.

CONCEPT → REAL EXAMPLE → RISK

Stop or reduce new work, allow permitted in-flight work to complete within... → An instance becomes unready, drains HTTP requests, finishes or safely abandons a... → A shutdown hook alone does not make accepted in-memory jobs durable

EXAMPLE FROM THE ORIGINAL GUIDE

An instance becomes unready, drains HTTP requests, finishes or safely abandons a claimed batch, and releases resources. Unfinished durable jobs can resume elsewhere after their claim expires.

IMPLEMENTATION SKETCH *illustrative*

```
server.shutdown=graceful
spring.lifecycle.timeout-per-shutdown-phase=20s
```

INTERVIEWER'S FOLLOW-UP

Which work must be recoverable if the process dies immediately?

Answer → Every uncommitted operation, accepted durable job, outbox intent and unacknowledged message must be retryable or reconciled after an immediate crash.

COMMON INTERVIEW TRAP: A shutdown hook alone does not make accepted in-memory jobs durable.

MY ONE-LINE RECAP _____

CHAPTER 20/24

Capacity, performance and regional design

Scale the constraint, not only the number of pods.

Five interview questions

Q096. How do you estimate concurrency and capacity?

Q097. Why might adding replicas fail to improve throughput?

Q098. How do virtual threads and reactive I/O affect microservices?

Q099. How should thread, connection and queue pools be tuned?

Q100. What makes multi-region microservices difficult?

Your challenge

Explain one correct operation, one failure path and the evidence that would tell you the design is working.

Capacity, performance and regional design

Q096. How do you estimate concurrency and capacity?

THE 30-SECOND ANSWER

Concurrency is approximately arrival rate times time in system for stable averages. Capacity depends on shared databases, thread pools, network and queues.

CONCEPT → REAL EXAMPLE → RISK

For a stable system, Little's Law relates average work in the system to...

→

At 500 requests per second and 0

→

This average is not a sufficient capacity limit or a p99 latency guarantee

EXAMPLE FROM THE ORIGINAL GUIDE

At 500 requests per second and 0.2 seconds average residence, average in-flight work is about 100. If residence rises to one second, the same stable rate implies about 500 in flight.

IMPLEMENTATION SKETCH *illustrative*

```
// Stable-load approximation (not a capacity guarantee):
inFlight = arrivalRatePerSec * avgLatencySeconds;
```

INTERVIEWER'S FOLLOW-UP

What happens if arrival rate exceeds sustainable service rate?

Answer → If incoming work exceeds sustainable completion rate, queued requests and latency grow until throttling, scaling, rejection or resource failure occurs.

COMMON INTERVIEW TRAP: This average is not a sufficient capacity limit or a p99 latency guarantee.

MY ONE-LINE RECAP _____

Capacity, performance and regional design

Q097. Why might adding replicas fail to improve throughput?

THE 30-SECOND ANSWER

More replicas do not help when all wait on one database, lock, broker partition or external quota.

CONCEPT → REAL EXAMPLE → RISK

Replicas help when application-instance capacity is the bottleneck and work... → Ten replicas each request 50 database connections against a database sized for 100... → Autoscaling must consider downstream capacity and hot-key behavior

EXAMPLE FROM THE ORIGINAL GUIDE

Ten replicas each request 50 database connections against a database sized for 100 useful concurrent connections. Scaling to 20 replicas increases waits rather than productive work.

INTERVIEWER'S FOLLOW-UP

Which metric would show that the database is the limiting resource?

Answer → DB CPU or locks at saturation, rising query/pool wait and flat successful transactions per second while pod count increases identify the bottleneck.

COMMON INTERVIEW TRAP: Autoscaling must consider downstream capacity and hot-key behavior.

MY ONE-LINE RECAP _____

Capacity, performance and regional design

Q098. How do virtual threads and reactive I/O affect microservices?

THE 30-SECOND ANSWER

Virtual threads make many blocking waits cheaper; reactive I/O uses nonblocking composition. Neither increases database or API capacity.

CONCEPT → REAL EXAMPLE → RISK

Virtual threads reduce the cost of many blocking tasks; reactive I/O... → A Java 21 service can handle many waiting requests with virtual threads, but its... → The Java 17 practice models here do not execute virtual-thread behavior

EXAMPLE FROM THE ORIGINAL GUIDE

A Java 21 service can handle many waiting requests with virtual threads, but its 30 database connections still admit only a limited amount of simultaneous database work.

IMPLEMENTATION SKETCH *illustrative*

```
// Virtual threads do not create more DB connections.
Semaphore dbBudget = new Semaphore(40);
```

INTERVIEWER'S FOLLOW-UP

When would a semaphore still be needed with virtual threads?

Answer → Use a semaphore when thousands of cheap virtual threads could otherwise open too many concurrent database, payment or other bounded dependency requests.

COMMON INTERVIEW TRAP: The Java 17 practice models here do not execute virtual-thread behavior.

MY ONE-LINE RECAP _____

Capacity, performance and regional design

Q099. How should thread, connection and queue pools be tuned?

THE 30-SECOND ANSWER

Tune pools from actual CPU, service time, connection availability, queue age and failure budget—not a copied universal number.

CONCEPT → REAL EXAMPLE → RISK

Treat them as connected resource budgets, not independent knobs to maximize	→ HTTP workers wait on database connections while the queue grows	→ A queue can improve burst handling while making deadline misses worse

EXAMPLE FROM THE ORIGINAL GUIDE

HTTP workers wait on database connections while the queue grows. Raising worker count alone creates more waiters. First identify slow queries, lock contention and the useful database concurrency budget.

IMPLEMENTATION SKETCH *illustrative*

```
maximumPoolSize = 12
// Multiply per-pod pools by max number of replicas.
```

INTERVIEWER'S FOLLOW-UP

Which measurements would you collect before changing a pool size?

Answer → Record active/idle/waiting counts, acquisition wait p95/p99, throughput, errors, CPU, lock contention, queue age and downstream limits first.

COMMON INTERVIEW TRAP: A queue can improve burst handling while making deadline misses worse.

MY ONE-LINE RECAP _____

Capacity, performance and regional design

Q100. What makes multi-region microservices difficult?

THE 30-SECOND ANSWER

Multi-region systems must handle replication delay, conflicting writes, regional identity, routing, cost and disaster recovery.

CONCEPT → REAL EXAMPLE → RISK

Decide write ownership, replication, failover authority, consistency and... → Catalog reads use local replicas, while payment operation identities remain under... → Two regions sharing one failure-prone dependency do not supply independent...

EXAMPLE FROM THE ORIGINAL GUIDE

Catalog reads use local replicas, while payment operation identities remain under an authoritative writer with controlled failover. Reconciliation detects late regional results after traffic moves.

IMPLEMENTATION SKETCH *illustrative*

```
// Two regions must not mint two charge identities.
operationKey = tenantId + ":" + clientPaymentId;
```

INTERVIEWER'S FOLLOW-UP

What happens if both regions believe they own the same payment operation?

Answer → If two regions issue the same payment operation independently, double charge is possible unless a global authority or provider-enforced idempotency key resolves ownership.

COMMON INTERVIEW TRAP: Two regions sharing one failure-prone dependency do not supply independent recovery.

MY ONE-LINE RECAP _____

CHAPTER 21/24

Migration and architecture evolution

Migrate data ownership before retiring the old path.

Five interview questions

Q101. What is the strangler-fig migration pattern?

Q102. What is an anti-corruption layer?

Q103. How do you extract a service without keeping shared-table coupling?

Q104. How do you perform a reliable backfill and cutover?

Q105. When should services be merged or a boundary changed?

Your challenge

Explain one correct operation, one failure path and the evidence that would tell you the design is working.

Migration and architecture evolution

Q101. What is the strangler-fig migration pattern?

THE 30-SECOND ANSWER

The strangler pattern routes selected capabilities away from a legacy system to new implementations over time. It needs careful ownership cutover.

CONCEPT → REAL EXAMPLE → RISK

Replace selected legacy capabilities gradually through a controlled routing... → New address-validation requests go to an extracted service, while remaining loan... → A supposedly temporary shared write path can become permanent coupling

EXAMPLE FROM THE ORIGINAL GUIDE

New address-validation requests go to an extracted service, while remaining loan functions stay in the legacy application. Compare outcomes and move traffic progressively before retiring the old validation path.

INTERVIEWER'S FOLLOW-UP

What objective determines when the legacy path can be removed?

Answer → Remove the old route when all clients use the new owner, reconciled data agrees, errors meet targets and rollback/recovery paths are proven.

COMMON INTERVIEW TRAP: A supposedly temporary shared write path can become permanent coupling.

MY ONE-LINE RECAP _____

Migration and architecture evolution

Q102. What is an anti-corruption layer?

THE 30-SECOND ANSWER

An anti-corruption layer translates a legacy model into the new domain language so old semantics do not spread through new services.

CONCEPT → REAL EXAMPLE → RISK

An anti-corruption layer translates another system's model into the local... → Legacy status code A7 maps to the origination domain's DOCUMENT_REVIEW_REQUIRED → Translation must preserve business meaning, not just rename fields

EXAMPLE FROM THE ORIGINAL GUIDE

Legacy status code A7 maps to the origination domain's DOCUMENT_REVIEW_REQUIRED. Local code uses its own meaningful state instead of distributing A7 checks across every handler.

INTERVIEWER'S FOLLOW-UP

Who owns changes when the legacy semantics evolve?

Answer → The integration/ACL owner tracks legacy contract changes, adds translation tests and negotiates meaning with the legacy team before updating downstream models.

COMMON INTERVIEW TRAP: Translation must preserve business meaning, not just rename fields.

MY ONE-LINE RECAP _____

Migration and architecture evolution

Q103. How do you extract a service without keeping shared-table coupling?

THE 30-SECOND ANSWER

Extract data ownership by changing writers/readers to use the new owner's contracts, not by simply copying shared tables.

CONCEPT → REAL EXAMPLE → RISK

Identify the capability and invariants, establish a contract, and move... → Inventory first owns reservation writes, then checkout stops direct stock updates → Extraction is incomplete while multiple systems independently enforce the...

EXAMPLE FROM THE ORIGINAL GUIDE

Inventory first owns reservation writes, then checkout stops direct stock updates. A compatibility adapter supports old callers during migration while the team audits and removes remaining database permissions.

IMPLEMENTATION SKETCH *illustrative*

```
// Replace other services' direct table writes:
legacyWriter -> ownerApi -> privateDatabase
```

INTERVIEWER'S FOLLOW-UP

How do you discover hidden writers and batch jobs?

Answer → Audit database users, triggers, stored procedures, scheduled jobs, reports, CDC subscriptions and application query logs to find all hidden writers.

COMMON INTERVIEW TRAP: Extraction is incomplete while multiple systems independently enforce the same invariant.

MY ONE-LINE RECAP _____

Migration and architecture evolution

Q104. How do you perform a reliable backfill and cutover?

THE 30-SECOND ANSWER

A backfill copies existing history while live writes continue. It needs a stable watermark and replay/catch-up phase before switching readers.

CONCEPT → REAL EXAMPLE → RISK

Record a clear change boundary, copy historical data, capture overlapping... → Build a new order projection from a consistent snapshot plus subsequent events → Equal row counts do not prove identical or correctly scoped data

EXAMPLE FROM THE ORIGINAL GUIDE

Build a new order projection from a consistent snapshot plus subsequent events. Compare tenant-level counts and accepted totals, catch up, then switch the query endpoint to the new projection.

INTERVIEWER'S FOLLOW-UP

Which checks detect missed updates and duplicate history?

Answer → Compare row counts, checksums, aggregate totals, IDs and source version watermarks; test overlap handling and duplicate CDC delivery.

COMMON INTERVIEW TRAP: Equal row counts do not prove identical or correctly scoped data.

MY ONE-LINE RECAP _____

Migration and architecture evolution

Q105. When should services be merged or a boundary changed?

THE 30-SECOND ANSWER

Merging services can reduce network chatter and release coupling when ownership boundaries no longer justify a remote call.

CONCEPT → REAL EXAMPLE → RISK

Revisit boundaries when common changes require constant coordination,... → Two tiny pricing services always release together and exchange many synchronous... → Do not merge an unrelated high-scale workload just to reduce service count

EXAMPLE FROM THE ORIGINAL GUIDE

Two tiny pricing services always release together and exchange many synchronous calls. One deployment with internal pricing modules removes a needless network boundary while retaining clear code ownership.

IMPLEMENTATION SKETCH *illustrative*

```
// Measure before and after: release lead time, p99,
// incident rate, per-request infrastructure cost.
```

INTERVIEWER'S FOLLOW-UP

Which before-and-after metrics would justify the change?

Answer → Compare deployment lead time, cross-service change frequency, p99, incident rate, infrastructure cost and team ownership before and after.

COMMON INTERVIEW TRAP: Do not merge an unrelated high-scale workload just to reduce service count.

MY ONE-LINE RECAP _____

CHAPTER 22/24

Business architecture case studies

Explain one happy path and its worst crash window.

Five interview questions

Q106. How would you design checkout and order acceptance?

Q107. How would you design a payment service?

Q108. How would you design a notification service?

Q109. How would you design loan origination microservices?

Q110. How would you prevent double booking in a reservation system?

Your challenge

Explain one correct operation, one failure path and the evidence that would tell you the design is working.

Business architecture case studies

Q106. How would you design checkout and order acceptance?

THE 30-SECOND ANSWER

Checkout coordinates Orders, Inventory and Payment through durable state transitions, stable operation IDs and recoverable commands.

CONCEPT → REAL EXAMPLE → RISK

Separate order identity, inventory reservation and payment authority	→ Order o-71 moves from ACCEPTED to RESERVED to PAYMENT_AUTHORIZED to CONFIRMED	→ Success is a business state, not simply an HTTP 200 from the orchestrator

EXAMPLE FROM THE ORIGINAL GUIDE

Order o-71 moves from ACCEPTED to RESERVED to PAYMENT_AUTHORIZED to CONFIRMED. A definitive payment rejection releases reservation r-8 and ends in REJECTED; an unknown payment outcome follows reconciliation instead.

IMPLEMENTATION SKETCH *illustrative*

```
ACCEPTED -> RESERVE -> PAYMENT_PENDING -> CONFIRMED
// Every state transition is durably recorded.
```

INTERVIEWER'S FOLLOW-UP

Where does the workflow store each transition and pending command?

Answer → Store each workflow state and pending outbox command in one local transaction; a relay publishes the command and a worker advances state conditionally.

COMMON INTERVIEW TRAP: Success is a business state, not simply an HTTP 200 from the orchestrator.

MY ONE-LINE RECAP _____

Business architecture case studies

Q107. How would you design a payment service?

THE 30-SECOND ANSWER

A payment service needs tenant-scoped operation IDs, an authoritative ledger, careful provider integration and reconciliation after unknown outcomes.

CONCEPT → REAL EXAMPLE → RISK

Model payment operations with tenant-scoped identities, canonical requests,... → Charge operation p-17 becomes PROVIDER_UNKNOWN after a transport timeout → Financial effects require provider-specific guarantees and reconciliation,...

EXAMPLE FROM THE ORIGINAL GUIDE

Charge operation p-17 becomes PROVIDER_UNKNOWN after a transport timeout. A callback with the matching provider reference confirms CAPTURED. A replay returns the recorded result rather than submitting a new charge.

IMPLEMENTATION SKETCH *illustrative*

```
UNIQUE (tenant_id, payment_operation_id)
// Callback replays reuse the same payment.
```

INTERVIEWER'S FOLLOW-UP

How do you handle duplicate or out-of-order provider callbacks?

Answer → Deduplicate callbacks using provider event/operation IDs, check expected state transitions and reject stale callbacks by sequence/version instead of overwriting final state.

COMMON INTERVIEW TRAP: Financial effects require provider-specific guarantees and reconciliation, beyond this teaching design.

MY ONE-LINE RECAP _____

Business architecture case studies

Q108. How would you design a notification service?

THE 30-SECOND ANSWER

Notifications should be accepted as durable jobs, processed asynchronously and retried with rate limits and a clear dead-letter policy.

CONCEPT → REAL EXAMPLE → RISK

Accept a durable notification intent with identity, recipient scope and…	→ OrderAccepted creates email job n-24 once	→ Some providers cannot guarantee one visible delivery; state that limitation…

EXAMPLE FROM THE ORIGINAL GUIDE

OrderAccepted creates email job n-24 once. A worker submits it with a stable provider operation key where supported. A lost response leads to recovery under the provider's contract, not an unconditional second email.

IMPLEMENTATION SKETCH *illustrative*

```
POST /notifications -> 202 + jobId
GET /notifications/{jobId} -> PENDING/SENT/FAILED
```

INTERVIEWER'S FOLLOW-UP

What does the API promise when it returns accepted?

Answer → HTTP 202 should mean a durable notification request was accepted, not delivered. Expose job status and delivery outcome or a documented best-effort contract.

COMMON INTERVIEW TRAP: Some providers cannot guarantee one visible delivery; state that limitation explicitly.

MY ONE-LINE RECAP _____

Business architecture case studies

Q109. How would you design loan origination microservices?

THE 30-SECOND ANSWER

Loan origination can separate application intake, offers, underwriting and partner integration with a durable case workflow and strict identity boundaries.

CONCEPT → REAL EXAMPLE → RISK

Use domain capabilities such as application intake, document handling,...

→

Application a-55 stays UNDERWRITING_PENDING while a bureau check is retried...

→

A credit decision and a disbursement have different authority and recovery...

EXAMPLE FROM THE ORIGINAL GUIDE

Application a-55 stays UNDERWRITING_PENDING while a bureau check is retried through a durable workflow. A completed decision records policy version and evidence references before disbursement becomes eligible.

IMPLEMENTATION SKETCH *illustrative*

```
APPLICATION_RECEIVED -> OFFER_SELECTED -> CHECK_PENDING
-> APPROVED / REJECTED / REVIEW_REQUIRED
```

INTERVIEWER'S FOLLOW-UP

How would you resume applications after a partner outage?

Answer → Save application state and partner request ID, display pending, retry only idempotent checks, and resume from the last committed step once the partner returns.

COMMON INTERVIEW TRAP: A credit decision and a disbursement have different authority and recovery requirements.

MY ONE-LINE RECAP _____

Business architecture case studies

Q110. How would you prevent double booking in a reservation system?

THE 30-SECOND ANSWER

Prevent double booking using one authoritative conditional reservation/write for a slot, with unique identity and lifecycle transitions.

CONCEPT → REAL EXAMPLE → RISK

Enforce the seat, room or slot invariant at the authoritative store through... → Seat s-9 is held under reservation r-2 until its expiry → A distributed cache lock alone may not protect the authoritative booking...

EXAMPLE FROM THE ORIGINAL GUIDE

Seat s-9 is held under reservation r-2 until its expiry. A confirmation succeeds only while r-2 is still the valid hold. If payment arrives late, reconcile payment and availability instead of silently granting the seat twice.

IMPLEMENTATION SKETCH *illustrative*

```
UPDATE seats SET state='CONFIRMED'
 WHERE id=:id AND state='HELD' AND version=:v;
```

INTERVIEWER'S FOLLOW-UP

How do expiry and confirmation avoid racing into two final states?

Answer → Use conditional version/state updates so only ACTIVE may become CONFIRMED or EXPIRED. The winning transaction defines the final state; retries return it.

COMMON INTERVIEW TRAP: A distributed cache lock alone may not protect the authoritative booking write.

MY ONE-LINE RECAP _____

CHAPTER 23/24

Production diagnosis and recovery

Collect evidence before retrying or replacing everything.

Five interview questions

Q111. How would you diagnose rising 504 responses?

Q112. How would you investigate duplicate business effects?

Q113. How would you diagnose a growing Kafka backlog?

Q114. Why might every replica restart during a dependency outage?

Q115. How would you investigate stale or cross-tenant cache results?

Your challenge

Explain one correct operation, one failure path and the evidence that would tell you the design is working.

Production diagnosis and recovery

Q111. How would you diagnose rising 504 responses?

THE 30-SECOND ANSWER

504s can come from queue waits, pool exhaustion, slow dependencies or improper deadlines. Trace before changing timeouts or adding retries.

CONCEPT → REAL EXAMPLE → RISK

Find where the deadline expires, then separate upstream queueing,... → Traces show 700 ms waiting for payment connections before a 300 ms remote call → The layer returning 504 may not be the layer causing delay

EXAMPLE FROM THE ORIGINAL GUIDE

Traces show 700 ms waiting for payment connections before a 300 ms remote call. Pool utilization and queue growth support a saturation diagnosis; a provider latency chart alone would miss it.

IMPLEMENTATION SKETCH *illustrative*

```
504 -> trace -> pool wait -> downstream latency
// Never create a fresh payment ID during repair.
```

INTERVIEWER'S FOLLOW-UP

What repair reduces waiting without increasing double-charge risk?

Answer → Reduce queues with bounded admission, isolate slow dependencies and recover unknown charges by stable operation ID. Never blindly retry using a new payment identity.

COMMON INTERVIEW TRAP: The layer returning 504 may not be the layer causing delay.

MY ONE-LINE RECAP _____

Production diagnosis and recovery

Q112. How would you investigate duplicate business effects?

THE 30-SECOND ANSWER

Duplicate effects often come from retries, broker redelivery or crashes between commit and acknowledgement. Trace the operation identity end-to-end.

CONCEPT → REAL EXAMPLE → RISK

Trace the stable operation or event identity across producer, relay,... → Two charges share one checkout ID but different payment operation IDs → Exactly-once broker processing does not cover new logical identities

EXAMPLE FROM THE ORIGINAL GUIDE

Two charges share one checkout ID but different payment operation IDs. The caller generated a new key after each timeout, so consumer deduplication alone cannot repair the cause.

IMPLEMENTATION SKETCH *illustrative*

```
// Regression: kill after ledger COMMIT, before ACK.
// Replay should not credit twice.
```

INTERVIEWER'S FOLLOW-UP

Which test should permanently reproduce the incident's crash window?

Answer → Keep a regression test that forces the exact crash after the business commit but before the acknowledgement, then replays and asserts one durable effect.

COMMON INTERVIEW TRAP: Exactly-once broker processing does not cover new logical identities.

MY ONE-LINE RECAP _____

Production diagnosis and recovery

Q113. How would you diagnose a growing Kafka backlog?

THE 30-SECOND ANSWER

Kafka lag rises when arrival exceeds throughput or one partition/message stalls. Check per-partition progress and downstream delays.

CONCEPT → REAL EXAMPLE → RISK

Compare production rate, processing rate, record age and per-partition lag	→ One partition contains a hot tenant and lags while five partitions are idle	→ Low committed-offset lag can hide an unbounded in-process task backlog

EXAMPLE FROM THE ORIGINAL GUIDE

One partition contains a hot tenant and lags while five partitions are idle. Additional group members cannot split that traditional partition assignment; key design or workload isolation needs review.

IMPLEMENTATION SKETCH *illustrative*

```
// Keep aggregate events on a stable partition key.
producer.send(topic, aggregateId, event);
```

INTERVIEWER'S FOLLOW-UP

How would you rebalance the workload without breaking order guarantees?

Answer → Use stable aggregate keys, partition-aware scaling and controlled handover or sequence gates; do not randomly redistribute events that require order.

COMMON INTERVIEW TRAP: Low committed-offset lag can hide an unbounded in-process task backlog.

MY ONE-LINE RECAP _____

Production diagnosis and recovery

Q114. Why might every replica restart during a dependency outage?

THE 30-SECOND ANSWER

A dependency outage can make readiness/liveness fail on all pods, triggering traffic withdrawal or restart storms.

CONCEPT → REAL EXAMPLE → RISK

A liveness policy may treat a shared downstream outage as a restart-worthy... → All catalog Pods fail liveness when recommendation is down, even though catalog... → Do not disable probes blindly; repair their failure model

EXAMPLE FROM THE ORIGINAL GUIDE

All catalog Pods fail liveness when recommendation is down, even though catalog can serve product data. Restarting catalog cannot fix recommendation and interrupts otherwise usable traffic.

IMPLEMENTATION SKETCH *illustrative*

```
// Do NOT make all downstream failures restart-worthy.
// Readiness and liveness answer different questions.
```

INTERVIEWER'S FOLLOW-UP

Which signal actually indicates that restarting this process may help?

Answer → Liveness should fail for a process-local condition a restart can repair, such as a permanently wedged server thread loop, not just remote service unavailability.

COMMON INTERVIEW TRAP: Do not disable probes blindly; repair their failure model.

MY ONE-LINE RECAP _____

Production diagnosis and recovery

Q115. How would you investigate stale or cross-tenant cache results?

THE 30-SECOND ANSWER

Stale or cross-tenant cache content can come from bad keys, unsafe invalidation or missing authorization. Treat a data leak as security-critical.

CONCEPT → REAL EXAMPLE → RISK

Check the exact cache key, trusted tenant context, permissions, value...

→

A product key omits tenant ID, allowing tenant B to read A's negotiated price

→

A global cache can magnify a local identity mistake across all replicas

EXAMPLE FROM THE ORIGINAL GUIDE

A product key omits tenant ID, allowing tenant B to read A's negotiated price. Adding a shorter TTL only shortens the leak; a scoped key and verified caller context fix the identity boundary.

IMPLEMENTATION SKETCH *illustrative*

```
cacheKey = verifiedTenantId + ":" + resourceId;
// Owner still checks permission on a miss.
```

INTERVIEWER'S FOLLOW-UP

Which test rejects the same resource ID under a different tenant?

Answer → Request the same order ID under a different authenticated tenant and assert a denial even if the first tenant populated the cache.

COMMON INTERVIEW TRAP: A global cache can magnify a local identity mistake across all replicas.

MY ONE-LINE RECAP _____

CHAPTER 24/24

Senior and architect interview judgment

Make the trade-off defendable and measurable.

Five interview questions

Q116. How should you structure a microservices design interview answer?

Q117. What should an architecture decision record contain?

Q118. How do you prove independent deployability?

Q119. What should a production release gate cover?

Q120. How do you balance architectural quality, cost and team capability?

Your challenge

Explain one correct operation, one failure path and the evidence that would tell you the design is working.

Senior and architect interview judgment

Q116. How should you structure a microservices design interview answer?

THE 30-SECOND ANSWER

Start by clarifying users, traffic, invariants, consistency and failure objectives. Then design an owner, happy path, crash path and verification plan.

CONCEPT → REAL EXAMPLE → RISK

Clarify business operations, invariants, load, latency, availability and... → For checkout, first define whether payment capture must precede confirmation and... → Naming Kafka, Redis and Kubernetes before the requirements weakens the design

EXAMPLE FROM THE ORIGINAL GUIDE

> For checkout, first define whether payment capture must precede confirmation and how long stock may be held. Those requirements determine workflow states, compensation and user-visible pending behavior.

INTERVIEWER'S FOLLOW-UP

Which assumption would change the architecture most?

Answer → A strict cross-region linearizable payment requirement, or a hard recovery target, may change data authority and topology more than a traffic estimate.

COMMON INTERVIEW TRAP: Naming Kafka, Redis and Kubernetes before the requirements weakens the design.

MY ONE-LINE RECAP _____

Senior and architect interview judgment

Q117. What should an architecture decision record contain?

THE 30-SECOND ANSWER

An ADR records context, decision, alternatives, tradeoffs, risks, evidence and when to revisit it—not just the chosen tool name.

CONCEPT → REAL EXAMPLE → RISK

Record the decision's context, constraints, considered options, chosen...	→ An ADR chooses orchestration for a five-step loan workflow because audit and...	→ A decision without a measurable consequence is difficult to evaluate later

EXAMPLE FROM THE ORIGINAL GUIDE

An ADR chooses orchestration for a five-step loan workflow because audit and recovery need one visible progress record. It also documents coordinator durability, team ownership and a review point if workflow volume changes.

INTERVIEWER'S FOLLOW-UP

What observation would make you reverse the decision?

Answer → Reverse it if measured lead time, latency, risk or operating cost violates the decision criteria, or the expected ownership benefit fails to appear.

COMMON INTERVIEW TRAP: A decision without a measurable consequence is difficult to evaluate later.

MY ONE-LINE RECAP _____

Senior and architect interview judgment

Q118. How do you prove independent deployability?

THE 30-SECOND ANSWER

Independent deployment means one service can release a compatible change without coordinated changes to others or shared data migrations.

CONCEPT → REAL EXAMPLE → RISK

Verify that compatible versions coexist across APIs, events and schemas,... → Deploy a new inventory version while old and new order instances remain active → A separate Git repository is not evidence of runtime or data compatibility

EXAMPLE FROM THE ORIGINAL GUIDE

Deploy a new inventory version while old and new order instances remain active. All supported reservation contracts pass; schema changes still allow rollback to the approved old inventory image.

INTERVIEWER'S FOLLOW-UP

Which release dependency would show that independence is still incomplete?

Answer → A release that requires consumers to deploy first, or requires a shared schema write used by another service, shows the dependency is not yet independent.

COMMON INTERVIEW TRAP: A separate Git repository is not evidence of runtime or data compatibility.

MY ONE-LINE RECAP _____

Senior and architect interview judgment

Q119. What should a production release gate cover?

THE 30-SECOND ANSWER

A release gate checks test evidence, compatibility, security, rollback, capacity, observability and explicit accepted risks.

CONCEPT → REAL EXAMPLE → RISK

Evaluate the concrete candidate's contract and schema compatibility,... → A release is blocked if a payment timeout can create a new charge identity or if... → A checklist cannot compensate for an untested critical invariant

EXAMPLE FROM THE ORIGINAL GUIDE

A release is blocked if a payment timeout can create a new charge identity or if the rollback image cannot read the migrated schema. A bounded recommendation degradation may be acceptable under the documented objective.

IMPLEMENTATION SKETCH *illustrative*

```
// Release gate: security + compatibility + restores +
// regression + rollback + production smoke test.
```

INTERVIEWER'S FOLLOW-UP

Which unresolved risk would you personally block, and why?

Answer → Block a release that may duplicate charges or leak one tenant's data without a tested control, because an ordinary rollback cannot reliably undo that effect.

COMMON INTERVIEW TRAP: A checklist cannot compensate for an untested critical invariant.

MY ONE-LINE RECAP _____

Senior and architect interview judgment

Q120. How do you balance architectural quality, cost and team capability?

THE 30-SECOND ANSWER

Choose architecture that meets business guarantees with skills and budget available. Smaller, cohesive services often beat fashionable complexity.

CONCEPT → REAL EXAMPLE → RISK

Optimize for the product's goals under real staffing, operational and cost...	→ A team keeps three cohesive services instead of twenty tiny ones, invests in...	→ More services can increase coordination and on-call load faster than...

EXAMPLE FROM THE ORIGINAL GUIDE

A team keeps three cohesive services instead of twenty tiny ones, invests in contract tests and outbox recovery, and measures release lead time, incident rate and useful throughput per cost.

INTERVIEWER'S FOLLOW-UP

Which simplification would preserve the business guarantees at lower cost?

Answer → Keep cohesive modules together, replace unnecessary synchronous hops with a local module call, or use a managed capability while preserving owned data contracts and recovery.

COMMON INTERVIEW TRAP: More services can increase coordination and on-call load faster than capability.

MY ONE-LINE RECAP _____

ARCHITECTURE REFERENCE

14 memory sketches

The reference diagrams on source pages 54–60 are redrawn as easy-to-follow handwritten notes. These diagrams illustrate particular boundaries and failure paths, not entire production systems.

> Sketch a case twice: first the successful path, then the crash window. Name which state is durable and who owns it.

D01

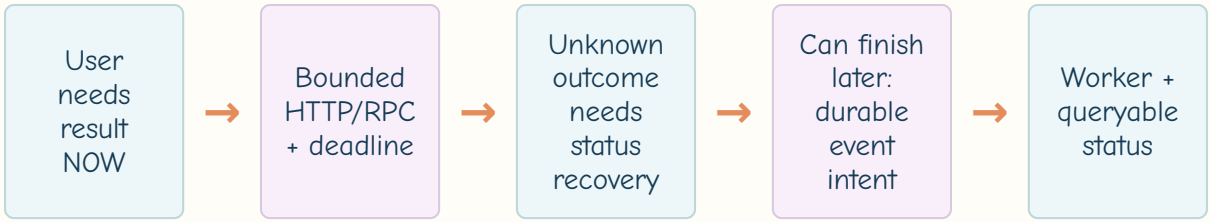
Service ownership and private data



Each business capability protects its own writes. Routing is not data authority.

D02

Immediate result versus deferred work



Async work adds durable pending state; synchronous calls still need uncertainty handling.

D03

One end-to-end deadline



Illustrative budget only; a local timeout cannot undo a committed remote operation.

D04

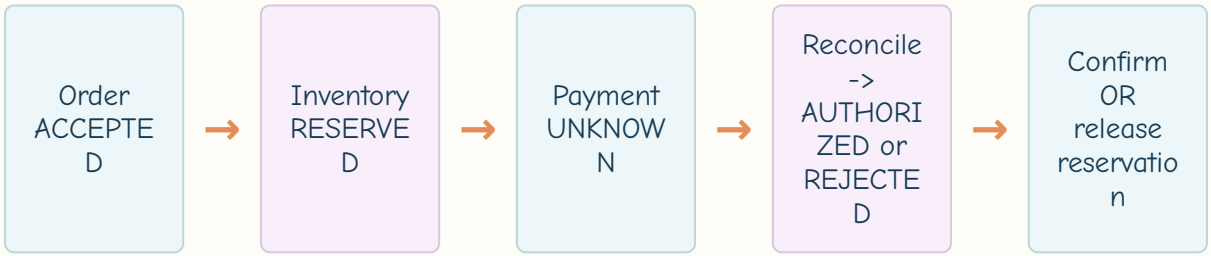
Isolate dependency concurrency



Separate admission budgets stop one slow optional dependency exhausting checkout.

D05

Saga with unknown outcome



Unknown means investigate rather than assuming the payment failed.

D06

Transactional outbox crash window



The outbox prevents missing intent; consumers still need deduplication.

D07

One consumer business effect



Duplicate event does not produce a duplicate local ledger effect.

D08

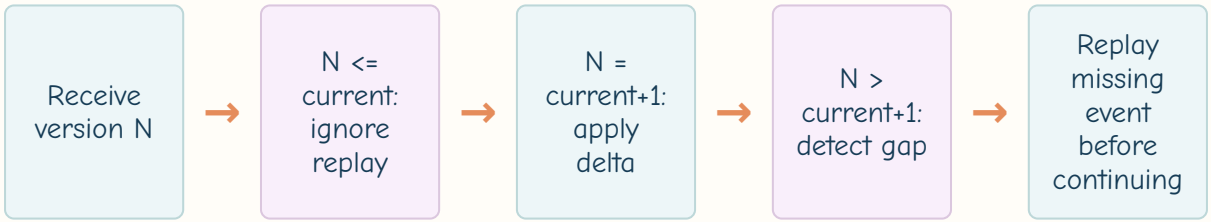
Kafka ordering is partition-scoped



Keep a given aggregate on the same partitioning policy and guard replay.

D09

Delta projection version gate



Persist version and delta together or risk a corrupt read model.

D10

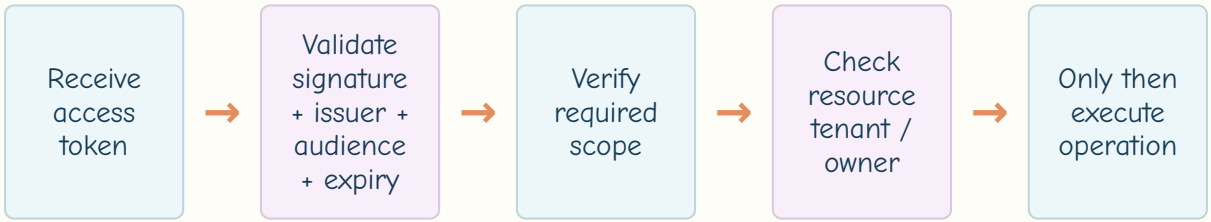
Cache identity includes tenant



Never let a shared resource ID alone pick another tenant's private response.

D11

JWT and domain authorization



Gateway authentication does not replace service-level object checks.

D12

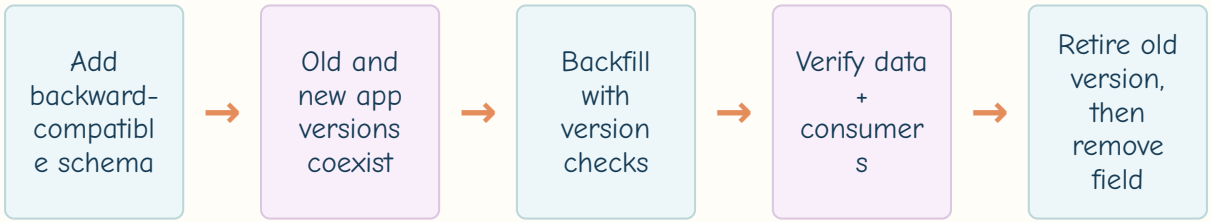
Trace across boundaries



A trace explains causality but is not the authoritative business operation key.

D13

Safe expand-contract release



Rollback of code cannot undo an unsafe destructive schema change.

D14

Startup, readiness and liveness



A remote failure is generally not a good local liveness condition.

PRACTICE LAB BOOK

12 applied exercises

The source guide describes 10 executable Java models and two real-boundary project recipes. The code pack contains a reconstructed, compiled Java 17 model class and source excerpts for PostgreSQL and Spring.

Local lab runner

```
javac --release 17 MicroservicesModels.java
java MicroservicesModels
```

30 focused teaching assertions passed in this edition. The source guide describes 7,061 checks for its larger original source pack, which is not available here to rerun.

Integration work

L11 PostgreSQL and L12 Spring HTTP/JWT remain recipes requiring your own application and live integration environment. Do not use them as a deployed production service.

Java model / re-tested in the local bundle

L01. Represent money in minor units

This is the original Java model fragment placed inside the attached teaching runner. The corresponding test checks are in the download bundle.

```
public record Money(String currency, long minorUnits) {
public Money {
Objects.requireNonNull(currency);
if (!currency.matches("[A-Z]{3}") || minorUnits < 0)
throw new IllegalArgumentException("invalid money");
}
public Money plus(Money other) {
if (!currency.equals(other.currency()))
throw new IllegalArgumentException("currency mismatch");
return new Money(currency,
Math.addExact(minorUnits, other.minorUnits()));
}
}
```

Practice task

Run the model with a valid example, a duplicate/conflict and a boundary failure. Explain which invariant the model checks and which distributed guarantee still needs a real datastore or broker.

Java model / re-tested in the local bundle

L02. Replay a scoped local calculation

This is the original Java model fragment placed inside the attached teaching runner. The corresponding test checks are in the download bundle.

```
public record OpKey(String tenant, String operation, String key) {
public OpKey {
nonBlank(tenant); nonBlank(operation); nonBlank(key);
}
}
static final class ReplayStore {
private record Entry(String fingerprint, String result) {}
private final Map<OpKey, Entry> entries = new HashMap<>();
synchronized String run(OpKey key, String fingerprint,
Supplier<String> calculation) {
Objects.requireNonNull(key); nonBlank(fingerprint);
Objects.requireNonNull(calculation);
Entry old = entries.get(key);
if (old != null) {
if (!old.fingerprint().equals(fingerprint))
throw new IllegalArgumentException("key conflict");
return old.result();
}
String value = Objects.requireNonNull(calculation.get());
entries.put(key, new Entry(fingerprint, value));
return value;
}
}
```

Practice task

Run the model with a valid example, a duplicate/conflict and a boundary failure. Explain which invariant the model checks and which distributed guarantee still needs a real datastore or broker.

Java model / re-tested in the local bundle

L03. Check a remaining deadline without overflow

This is the original Java model fragment placed inside the attached teaching runner. The corresponding test checks are in the download bundle.

```
static final class Deadline {
private final LongSupplier clock;
private final long started, budget;
Deadline(Duration duration, LongSupplier clock) {
this.clock = Objects.requireNonNull(clock);
this.budget = duration.toNanos();
if (budget <= 0) throw new IllegalArgumentException("budget");
this.started = clock.getAsLong();
}
long remainingNanos() {
long elapsed = clock.getAsLong() - started;
if (elapsed < 0) throw new IllegalStateException("clock");
return elapsed >= budget ? 0 : budget - elapsed;
}
boolean canFit(long workNanos, long delayNanos) {
if (workNanos <= 0 || delayNanos < 0)
throw new IllegalArgumentException("attempt");
long remaining = remainingNanos();
return delayNanos <= remaining
&& workNanos <= remaining - delayNanos;
}
}
```

Practice task

Run the model with a valid example, a duplicate/conflict and a boundary failure. Explain which invariant the model checks and which distributed guarantee still needs a real datastore or broker.

Java model / re-tested in the local bundle

L04. Plan capped backoff with full jitter

This is the original Java model fragment placed inside the attached teaching runner. The corresponding test checks are in the download bundle.

```
static long jitterDelayMs(int retryNumber, long baseMs,
long capMs, double randomUnit) {
if (retryNumber < 1 || retryNumber > 1000 || baseMs < 1
|| capMs < baseMs || capMs > 60_000
|| !(randomUnit >= 0 && randomUnit < 1))
throw new IllegalArgumentException("retry policy");
long ceiling = baseMs;
for (int i = 1; i < retryNumber && ceiling < capMs; i++) {
ceiling = ceiling >= capMs - ceiling
? capMs : ceiling * 2;
}
return (long) (randomUnit * ceiling);
}
```

Practice task

Run the model with a valid example, a duplicate/conflict and a boundary failure. Explain which invariant the model checks and which distributed guarantee still needs a real datastore or broker.

Java model / re-tested in the local bundle

L05. Reserve stock with an optimistic version

This is the original Java model fragment placed inside the attached teaching runner. The corresponding test checks are in the download bundle.

```
enum ReserveResult { RESERVED, VERSION_CONFLICT, INSUFFICIENT }
public record Stock(long version, int available) {}
static final class Inventory {
private final AtomicReference<Stock> state;
Inventory(int initial) {
if (initial < 0) throw new IllegalArgumentException("stock");
state = new AtomicReference<>(new Stock(0, initial));
}
Stock snapshot() { return state.get(); }
ReserveResult reserve(long expectedVersion, int quantity) {
if (quantity <= 0) throw new IllegalArgumentException("quantity");
Stock old = state.get();
if (old.version() != expectedVersion)
return ReserveResult.VERSION_CONFLICT;
if (old.available() < quantity)
return ReserveResult.INSUFFICIENT;
Stock next = new Stock(Math.addExact(old.version(), 1),
old.available() - quantity);
return state.compareAndSet(old, next)
? ReserveResult.RESERVED : ReserveResult.VERSION_CONFLICT;
}
}
```

Practice task

Run the model with a valid example, a duplicate/conflict and a boundary failure. Explain which invariant the model checks and which distributed guarantee still needs a real datastore or broker.

Java model / re-tested in the local bundle

L06. Validate saga state transitions

This is the original Java model fragment placed inside the attached teaching runner. The corresponding test checks are in the download bundle.

```
enum SagaState {
ACCEPTED, RESERVED, PAYMENT_UNKNOWN, AUTHORIZED,
CONFIRMED, REJECTED
}
static final class Workflow {
private SagaState state = SagaState.ACCEPTED;
synchronized SagaState state() { return state; }
synchronized boolean move(SagaState expected, SagaState next) {
Objects.requireNonNull(expected); Objects.requireNonNull(next);
if (state == next) return false;
if (state != expected) throw new IllegalStateException("stale state");
boolean allowed = switch (state) {
case ACCEPTED -> next == SagaState.RESERVED
|| next == SagaState.REJECTED;
case RESERVED -> next == SagaState.PAYMENT_UNKNOWN
|| next == SagaState.AUTHORIZED || next == SagaState.REJECTED;
case PAYMENT_UNKNOWN -> next == SagaState.AUTHORIZED
|| next == SagaState.REJECTED;
case AUTHORIZED -> next == SagaState.CONFIRMED;
case CONFIRMED, REJECTED -> false;
};
if (!allowed) throw new IllegalStateException("invalid transition");
state = next;
return true;
}
}
```

Practice task

Run the model with a valid example, a duplicate/conflict and a boundary failure. Explain which invariant the model checks and which distributed guarantee still needs a real datastore or broker.

Java model / re-tested in the local bundle

L07. Model business data and outbox intent together

This is the original Java model fragment placed inside the attached teaching runner. The corresponding test checks are in the download bundle.

```
public record OutboxEvent(String eventId, String orderId) {}
static final class OutboxModel {
private final Set<String> orders = new HashSet<>();
private final Map<String, OutboxEvent> outbox = new LinkedHashMap<>();
synchronized void accept(String orderId, String eventId) {
nonBlank(orderId); nonBlank(eventId);
if (orders.contains(orderId) || outbox.containsKey(eventId))
throw new IllegalArgumentException("duplicate identity");
OutboxEvent event = new OutboxEvent(eventId, orderId);
orders.add(orderId);
outbox.put(eventId, event);
}
synchronized Set<String> orders() { return Set.copyOf(orders); }
synchronized List<OutboxEvent> events() {
return List.copyOf(outbox.values());
}
}
```

Practice task

Run the model with a valid example, a duplicate/conflict and a boundary failure. Explain which invariant the model checks and which distributed guarantee still needs a real datastore or broker.

Java model / re-tested in the local bundle

L08. Deduplicate a local ledger credit

This is the original Java model fragment placed inside the attached teaching runner. The corresponding test checks are in the download bundle.

```
public record SeenKey(String consumer, String eventId) {}
static final class LedgerModel {
private long balance;
private final Map<SeenKey, Long> seen = new HashMap<>();
synchronized boolean credit(String consumer, String eventId,
long amount) {
nonBlank(consumer); nonBlank(eventId);
if (amount <= 0) throw new IllegalArgumentException("amount");
SeenKey key = new SeenKey(consumer, eventId);
Long previous = seen.get(key);
if (previous != null) {
if (previous.longValue() != amount)
throw new IllegalArgumentException("changed event");
return false;
}
long next = Math.addExact(balance, amount);
seen.put(key, amount);
balance = next;
return true;
}
synchronized long balance() { return balance; }
}
```

Practice task

Run the model with a valid example, a duplicate/conflict and a boundary failure. Explain which invariant the model checks and which distributed guarantee still needs a real datastore or broker.

Java model / re-tested in the local bundle

L09. Apply an ordered delta projection

This is the original Java model fragment placed inside the attached teaching runner. The corresponding test checks are in the download bundle.

```
public record View(long version, long total) {}
static final class Projection {
private final Map<String, View> views = new HashMap<>();
synchronized boolean apply(String aggregate, long version, long delta) {
nonBlank(aggregate);
if (version < 1) throw new IllegalArgumentException("version");
View old = views.getOrDefault(aggregate, new View(0, 0));
if (version <= old.version()) return false;
if (version != old.version() + 1)
throw new IllegalStateException("version gap");
View next = new View(version, Math.addExact(old.total(), delta));
views.put(aggregate, next);
return true;
}
synchronized View get(String aggregate) {
return views.getOrDefault(aggregate, new View(0, 0));
}
}
```

Practice task

Run the model with a valid example, a duplicate/conflict and a boundary failure. Explain which invariant the model checks and which distributed guarantee still needs a real datastore or broker.

Java model / re-tested in the local bundle

L10. Reject excess concurrent dependency work

This is the original Java model fragment placed inside the attached teaching runner. The corresponding test checks are in the download bundle.

```
static final class Bulkhead {
private final Semaphore permits;
Bulkhead(int limit) {
if (limit < 1) throw new IllegalArgumentException("limit");
permits = new Semaphore(limit);
}
<T> T call(Callable<T> task) throws Exception {
Objects.requireNonNull(task);
if (!permits.tryAcquire())
throw new RejectedExecutionException("bulkhead full");
try {
return task.call();
} finally {
permits.release();
}
}
}
```

Practice task

Run the model with a valid example, a duplicate/conflict and a boundary failure. Explain which invariant the model checks and which distributed guarantee still needs a real datastore or broker.

L11. Write an order and outbox in one SQL transaction

Adapt to your project; this original source recipe was not executed against the required database and Spring dependencies.

```
CREATE TABLE ms_order (
       tenant_id TEXT NOT NULL,
       order_id TEXT NOT NULL,
       state TEXT NOT NULL,
       PRIMARY KEY (tenant_id, order_id)
   );


   CREATE TABLE ms_outbox (
       event_id TEXT PRIMARY KEY,
       tenant_id TEXT NOT NULL,
       aggregate_id TEXT NOT NULL,
       aggregate_version BIGINT NOT NULL CHECK (aggregate_version > 0),
       payload JSONB NOT NULL,
       created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
       published_at TIMESTAMPTZ,
       UNIQUE (tenant_id, aggregate_id, aggregate_version),
       FOREIGN KEY (tenant_id, aggregate_id)
           REFERENCES ms_order (tenant_id, order_id)
   );


   BEGIN;
   INSERT INTO ms_order (tenant_id, order_id, state)
   VALUES ('tenant-a', 'o-71', 'ACCEPTED');


   INSERT INTO ms_outbox
       (event_id, tenant_id, aggregate_id, aggregate_version, payload)
   VALUES
       ('e-90', 'tenant-a', 'o-71', 1,
        '{"type":"OrderAccepted","orderId":"o-71"}');
   COMMIT;
```

Verification boundary

Run in a disposable authorized environment with matching versions, then verify transaction rollback, auth failures, timeouts and safe retry outcomes. Passing Java-only models does not validate this integration.

L12. Build a bounded Spring HTTP client and JWT policy

Adapt to your project; this original source recipe was not executed against the required database and Spring dependencies.

```
record ReserveCommand(String orderId, int quantity) {}
   record ReservationResponse(String reservationId, String state) {}


   @HttpExchange("/inventory")
   interface InventoryHttpApi {
       @PostExchange("/reservations")
       ReservationResponse reserve(
           @RequestHeader("Idempotency-Key") String operationKey,
           @RequestBody ReserveCommand command);
   }


   @Bean
   InventoryHttpApi inventoryApi(RestClient.Builder builder) {
       var transport = HttpClient.newBuilder()
           .connectTimeout(Duration.ofMillis(300)).build();
       var factory = new JdkClientHttpRequestFactory(transport);
       factory.setReadTimeout(Duration.ofMillis(700));
       var client = builder.baseUrl("https://inventory.example.com")
           .requestFactory(factory).build();
       return HttpServiceProxyFactory
           .builderFor(RestClientAdapter.create(client))
           .build().createClient(InventoryHttpApi.class);
   }
```

Verification boundary

Run in a disposable authorized environment with matching versions, then verify transaction rollback, auth failures, timeouts and safe retry outcomes. Passing Java-only models does not validate this integration.

Issuer + audience + domain permission

The original L12 continues on source page 74. These are example properties and still require real issuer keys, scopes and tenant/object checks.

```
spring:
     security:
       oauth2:
         resourceserver:
           jwt:
             issuer-uri: https://identity.example.com/issuer
             audiences: https://inventory.example.com
   management:
     endpoint:
       health:
         probes:
           enabled: true
```

Required testing

Reject missing/expired tokens; reject wrong audience or issuer; enforce scope; deny one tenant reading another tenant's resource. Confirm real client connection and response timeouts.

SENIOR REVISION

Pick the mechanism by the problem

Unknown remote outcome	Stable operation key + status lookup and reconciliation
Unsafe transient retry	End-to-end deadline, bounded backoff, jitter and stable key
Dependency repeats failure	Circuit breaker with timeout and admission controls
Too much in-flight work	Bulkhead/concurrency limit and bounded queue
Lost event after local commit	Transactional outbox, relay monitoring
Duplicate consumer effect	Processed event key and business effect in one transaction
Cross-service business workflow	Saga states, compensation and recovery
Stale read model	Projection lag + replay and version checks
Unsafe release	Contract verification and expand-contract schema

30-DAY INTERVIEW SPRINT

Study & practise / 1–15

DAY 01	Microservices foundations — explain five Q&As, draw the owner boundary and answer all follow-ups aloud.
DAY 02	Domain boundaries and ownership — explain five Q&As, draw the owner boundary and answer all follow-ups aloud.
DAY 03	REST contracts and API evolution — explain five Q&As, draw the owner boundary and answer all follow-ups aloud.
DAY 04	Communication and Java clients — explain five Q&As, draw the owner boundary and answer all follow-ups aloud.
DAY 05	Discovery, gateways and service mesh — explain five Q&As, draw the owner boundary and answer all follow-ups aloud.
DAY 06	Spring Boot implementation and configuration — explain five Q&As, draw the owner boundary and answer all follow-ups aloud.
DAY 07	Timeouts, deadlines and retries — explain five Q&As, draw the owner boundary and answer all follow-ups aloud.
DAY 08	Resilience and admission control — explain five Q&As, draw the owner boundary and answer all follow-ups aloud.
DAY 09	Data ownership and consistency — explain five Q&As, draw the owner boundary and answer all follow-ups aloud.
DAY 10	Concurrent writes and data invariants — explain five Q&As, draw the owner boundary and answer all follow-ups aloud.
DAY 11	Sagas and workflow recovery — explain five Q&As, draw the owner boundary and answer all follow-ups aloud.
DAY 12	Transactional outbox and idempotent consumers — explain five Q&As, draw the owner boundary and answer all follow-ups aloud.
DAY 13	Events, schemas and Kafka topics — explain five Q&As, draw the owner boundary and answer all follow-ups aloud.
DAY 14	Delivery semantics, consumer groups and replay — explain five Q&As, draw the owner boundary and answer all follow-ups aloud.
DAY 15	CQRS, projections and caching — explain five Q&As, draw the owner boundary and answer all follow-ups aloud.

30-DAY INTERVIEW SPRINT

Study & practise / 16–30

DAY 16	Security across service boundaries: one happy path, one crash path and five question follow-ups.
DAY 17	Observability and reliability objectives: one happy path, one crash path and five question follow-ups.
DAY 18	Testing distributed contracts and failures: one happy path, one crash path and five question follow-ups.
DAY 19	Deployment, Kubernetes and schema changes: one happy path, one crash path and five question follow-ups.
DAY 20	Capacity, performance and regional design: one happy path, one crash path and five question follow-ups.
DAY 21	Migration and architecture evolution: one happy path, one crash path and five question follow-ups.
DAY 22	Business architecture case studies: one happy path, one crash path and five question follow-ups.
DAY 23	Production diagnosis and recovery: one happy path, one crash path and five question follow-ups.
DAY 24	Senior and architect interview judgment: one happy path, one crash path and five question follow-ups.
DAY 25	Implement and explain Java models L01-L05
DAY 26	Implement and explain Java models L06-L10
DAY 27	Walk through PostgreSQL outbox with a rollback test
DAY 28	Walk through Spring client/JWT policy and forbidden-tenant tests
DAY 29	Mock design: checkout, payment, inventory, saga and recovery
DAY 30	Mock incident: 504s, lag, timeouts and safe retry

BEFORE AN INTERVIEW

Final 60-second checklist

Define the invariant

Who owns data? What must never be duplicated or lost? What state can a user see while work is pending?

Walk through one failure

What if the request times out after the remote effect succeeded? What if an outbox relay crashes after send? Can a consumer replay safely?

Prove the recovery

Which database constraint, durable event, stable identity or reconciliation process makes it correct? Which real integration test shows this?

Include the budget

What happens when requests, tasks or messages arrive faster than they finish? How are retries, deadlines, memory and connections bounded?

Tell the truth about verification

Passing local models proves local behavior. PostgreSQL, Kafka, Kubernetes, Spring Security and cloud recovery each require their own evidence.